Kubernetes Go Client (client-go) — Developer Book

A comprehensive, chapter-wise guide to interacting with the Kubernetes API using Go's official `client-go` library — from first connection to production-grade controllers, informers, and custom operators.

Table of Contents

#	Chapter	Topics Covered
01	**Introduction**	What is client-go, architecture, when to use it, Go vs other clients
02	**Installation & Setup**	go mod, client-go packages, kubeconfig, code generation
03	**Connecting to a Cluster**	InClusterConfig, kubeconfig, multi-cluster, REST config
04	**Working with Pods**	List, get, create, delete, logs, exec, watch pods
05	**Deployments & ReplicaSets**	Create, scale, rolling update, rollback, status
06	**Services & Networking**	ClusterIP, NodePort, LoadBalancer, endpoints
07	**ConfigMaps & Secrets**	Create, read, update, delete config and secrets
08	**Persistent Volumes & Storage**	PV, PVC, StorageClass management
09	**Namespaces & RBAC**	Namespaces, Roles, ClusterRoles, Bindings, ServiceAccounts
10	**Ingress**	Create and manage Ingress rules, TLS, annotations
11	**StatefulSets & DaemonSets**	Stateful workloads, node-level agents
12	**Jobs & CronJobs**	One-off tasks, scheduled jobs, parallel batch

#	Chapter	Topics Covered
13	**Informers & Watching**	SharedInformer, event handlers, workqueues, cache
14	**Custom Resources (CRDs)**	Dynamic client, typed CRD client, code-gen
15	**Error Handling & Retry Logic**	k8s errors, retry, backoff, conflict resolution
16	**Production Patterns**	Controller pattern, leader election, operator, multi-cluster

---

How to Read This Book

If you're brand new to client-go: Read 01 → 02 → 03 → 04 in order.

If you're building a deployment tool: Read 05 → 06 → 07 → 10.

If you're writing a Kubernetes controller/operator: Read 13 → 14 → 15 → 16.

If you're managing RBAC programmatically: Read 09 → 15.

If something broke: Jump to Chapter 15 (Error Handling).

Quick Reference

```go
import (
    "context"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

// Build client from kubeconfig
config, _ := clientcmd.BuildConfigFromFlags("", "/home/user/.kube/config")
clientset, _ := kubernetes.NewForConfig(config)

// List pods
pods, _ := clientset.CoreV1().Pods("default").List(context.TODO(),
```

```
    metav1.ListOptions{})
    for _, p := range pods.Items {
        fmt.Println(p.Name)
    }
```

Key Packages

Package	Import Path	Purpose
`kubernetes`	`k8s.io/client-go/kubernetes`	Main typed clientset
`clientcmd`	`k8s.io/client-go/tools/clientcmd`	Load kubeconfig
`rest`	`k8s.io/client-go/rest`	REST config, in-cluster
`cache`	`k8s.io/client-go/tools/cache`	Informers, listers
`workqueue`	`k8s.io/client-go/util/workqueue`	Rate-limiting queue
`retry`	`k8s.io/client-go/util/retry`	RetryOnConflict
`dynamic`	`k8s.io/client-go/dynamic`	Unstructured / CRD client
`corev1`	`k8s.io/api/core/v1`	Pod, Service, ConfigMap types
`appsv1`	`k8s.io/api/apps/v1`	Deployment, StatefulSet types
`metav1`	`k8s.io/apimachinery/pkg/apis/meta/v1`	ObjectMeta, ListOptions

Chapter 1: Introduction to client-go

What is client-go?

client-go is the official Go client library for the Kubernetes API, maintained by the Kubernetes project itself. It is the foundation for:

- `kubectl` (the Kubernetes CLI)
- `kube-controller-manager` (built-in controllers)
- Every major Kubernetes operator and controller

```
client-go is to Kubernetes what the AWS SDK is to AWS.
```

Why client-go Over Other Options?

Option	Pros	Cons
`client-go`	Official, full-featured, informers, workqueue	Verbose, steep learning curve
`controller-runtime`	Higher-level, operator-friendly	Abstraction hides internals
Raw HTTP	Total control	No type safety, lots of boilerplate
`kubectl` CLI	Simple	No programmatic control

> **Rule:** Use `client-go` directly when building low-level controllers or need full control. Use `controller-runtime` (via kubebuilder) for production operators.

---

client-go Architecture

```
Your Go Program
      │
      ├── kubernetes.Clientset       ← typed API access
      │       ├── CoreV1()           ← Pods, Services, ConfigMaps
      │       ├── AppsV1()           ← Deployments, StatefulSets
      │       ├── BatchV1()          ← Jobs, CronJobs
      │       ├── NetworkingV1()     ← Ingress, NetworkPolicy
      │       └── RbacV1()           ← Roles, Bindings
      │
      ├── dynamic.Interface          ← unstructured / CRD access
```

```
        │
        ├── cache.SharedInformer       ← watch + local cache
        │       └── workqueue          ← rate-limited event queue
        │
        └── rest.Config                ← connection & auth
```

API Groups and Versions in Go

API Group	Go Client Method	Resources
Core (`v1`)	`clientset.CoreV1()`	Pod, Service, ConfigMap, Secret, PV, PVC, Namespace, Node
Apps (`apps/v1`)	`clientset.AppsV1()`	Deployment, StatefulSet, DaemonSet, ReplicaSet
Batch (`batch/v1`)	`clientset.BatchV1()`	Job, CronJob
Networking	`clientset.NetworkingV1()`	Ingress, NetworkPolicy
RBAC	`clientset.RbacV1()`	Role, ClusterRole, RoleBinding, ClusterRoleBinding
Storage	`clientset.StorageV1()`	StorageClass, VolumeAttachment

The Desired State Model (Reconciliation)

client-go is built around the Kubernetes control-loop model:

```
┌──────────────────────────────────────────┐
│            Reconciliation Loop           │
│                                          │
│  1. Observe  →  Watch API for events     │
│  2. Diff     →  Desired state vs actual  │
```

```
  |  3. Act      →  Create / Update / Delete     |
  |  4. Repeat                                   |
  └──────────────────────────────────────────────┘
```

Your controller:

1. Uses an **Informer** to watch resources (cached locally)
2. Puts events into a **WorkQueue**
3. A **worker goroutine** dequeues and calls your **Reconcile** function
4. Reconcile compares desired vs actual and acts

What You Can Do with client-go

- **CRUD on any resource** — Pods, Deployments, Services, Secrets, CRDs
- **Watch resources** — real-time event streaming with Informers
- **Leader election** — only one controller instance acts at a time
- **Custom controllers** — reconcile any resource type
- **Exec into pods** — run commands programmatically
- **Port forwarding** — forward cluster ports to local
- **Certificate operations** — create CSRs, approve certificates

---

Go vs Other Language Clients

Feature	Go (client-go)	Python	PHP
Official support	Yes (core team)	Yes	Community
Informer / cache	Full support	Partial	No
Operator frameworks	controller-runtime, kubebuilder	kopf	None
Concurrency	Goroutines (excellent)	asyncio	Limited
Type safety	Full generated types	Yes	No
Performance	Best	Good	Good

Feature	Go (client-go)	Python	PHP
Use in production operators	Standard	Used	Rare

Real-World Use Cases

- **kubectl itself** — built on client-go
- **cert-manager** — manages TLS certificates as a controller
- **ArgoCD** — GitOps controller built with client-go
- **Prometheus Operator** — manages Prometheus instances
- **Custom CI/CD** — deploy apps, watch rollout status
- **Multi-tenant platforms** — provision namespaces, RBAC, quotas per customer
- **Cluster maintenance tools** — drain nodes, evict pods, rotate secrets

Chapter 2: Installation & Setup

Prerequisites

- Go 1.21+
- A running Kubernetes cluster (Minikube, kind, k3s, or cloud)
- kubectl configured (for getting credentials)

Initialize a Go Module

```
mkdir my-k8s-tool && cd my-k8s-tool
go mod init github.com/yourname/my-k8s-tool
```

Installing client-go

```
# Install client-go (match version to your cluster's Kubernetes version)
go get k8s.io/client-go@v0.29.0

# Core API types (Pod, Deployment, etc.)
go get k8s.io/api@v0.29.0

# Meta types (ObjectMeta, ListOptions, etc.)
go get k8s.io/apimachinery@v0.29.0

# Tidy up
go mod tidy
```

Version Compatibility

Kubernetes Cluster	client-go version
1.29.x	v0.29.x
1.28.x	v0.28.x
1.27.x	v0.27.x
1.26.x	v0.26.x

```
# Check your cluster version
kubectl version --short
```

Minimal go.mod

go

```
module github.com/yourname/my-k8s-tool

go 1.21

require (
    k8s.io/api            v0.29.0
    k8s.io/apimachinery   v0.29.0
    k8s.io/client-go      v0.29.0
)
```

Project Layout

```
my-k8s-tool/
├── go.mod
├── go.sum
├── main.go
├── pkg/
│   ├── client/
│   │   └── client.go       ← client initialization
│   ├── controller/
│   │   └── controller.go   ← reconciliation logic
│   └── util/
│       └── wait.go         ← helper functions
└── config/
    └── rbac.yaml           ← ServiceAccount + Role for in-cluster
```

Minimal main.go

go

```go
package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "path/filepath"

    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
)

func main() {
    // Load kubeconfig from default location
    kubeconfig := filepath.Join(os.Getenv("HOME"), ".kube", "config")
    config, err := clientcmd.BuildConfigFromFlags("", kubeconfig)
    if err != nil {
        log.Fatalf("Error building kubeconfig: %v", err)
    }
```

```
    // Create the clientset
    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        log.Fatalf("Error creating clientset: %v", err)
    }

    // List pods in default namespace
    pods, err := clientset.CoreV1().Pods("default").List(context.TODO(),
metav1.ListOptions{})
    if err != nil {
        log.Fatalf("Error listing pods: %v", err)
    }

    fmt.Printf("Found %d pods in 'default' namespace:\n", len(pods.Items))
    for _, pod := range pods.Items {
        fmt.Printf("  - %s (%s)\n", pod.Name, pod.Status.Phase)
    }
}
```

```
go run main.go
```

Key Import Paths

```go
import (
    // Client
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/rest"
    "k8s.io/client-go/dynamic"

    // Informers
    "k8s.io/client-go/informers"
    "k8s.io/client-go/tools/cache"
    "k8s.io/client-go/util/workqueue"

    // Retry
    "k8s.io/client-go/util/retry"

    // API types
```

```
    corev1  "k8s.io/api/core/v1"
    appsv1  "k8s.io/api/apps/v1"
    batchv1 "k8s.io/api/batch/v1"
    netv1   "k8s.io/api/networking/v1"
    rbacv1  "k8s.io/api/rbac/v1"

    // Meta types
    metav1  "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/api/errors"
    "k8s.io/apimachinery/pkg/api/resource"
    "k8s.io/apimachinery/pkg/util/intstr"
    "k8s.io/apimachinery/pkg/runtime/schema"
)
```

Fake Client for Testing

go

```
import (
    "k8s.io/client-go/kubernetes/fake"
    corev1 "k8s.io/api/core/v1"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

func TestMyController(t *testing.T) {
    // Create a fake clientset pre-seeded with a pod
    pod := &corev1.Pod{
        ObjectMeta: metav1.ObjectMeta{
            Name:      "test-pod",
            Namespace: "default",
        },
    }
    fakeClient := fake.NewSimpleClientset(pod)

    // Use fakeClient like a real clientset
    pods, err := fakeClient.CoreV1().Pods("default").List(context.TODO(),
metav1.ListOptions{})
    assert.NoError(t, err)
    assert.Equal(t, 1, len(pods.Items))
}
```

Environment Variables

```go
// Useful env vars for configuring client behavior
os.Getenv("KUBECONFIG")             // custom kubeconfig path
os.Getenv("KUBERNETES_SERVICE_HOST") // set when running in-cluster
os.Getenv("KUBERNETES_SERVICE_PORT") // set when running in-cluster
```

Build & Run

```
# Run locally
go run main.go

# Build binary
go build -o k8s-tool ./...

# Cross-compile for Linux (for Docker)
GOOS=linux GOARCH=amd64 go build -o k8s-tool-linux ./...

# Docker image
docker build -t myrepo/k8s-tool:latest .
docker push myrepo/k8s-tool:latest
```

Minimal Dockerfile

```dockerfile
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /k8s-tool ./...

FROM gcr.io/distroless/static:nonroot
COPY --from=builder /k8s-tool /k8s-tool
ENTRYPOINT ["/k8s-tool"]
```

Chapter 3: Connecting to a Cluster

rest.Config — The Foundation

All client-go connections start with a `*rest.Config`. Everything else builds on top of it.

```go
import (
    "k8s.io/client-go/rest"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/kubernetes"
)
```

Method 1: kubeconfig File (Local Development)

```go
import (
    "os"
    "path/filepath"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/kubernetes"
)

func newClientFromKubeconfig() (*kubernetes.Clientset, error) {
    home, _ := os.UserHomeDir()
    kubeconfig := filepath.Join(home, ".kube", "config")

    config, err := clientcmd.BuildConfigFromFlags("", kubeconfig)
    if err != nil {
        return nil, err
    }

    return kubernetes.NewForConfig(config)
}
```

Custom kubeconfig path from flag

```go
import "flag"

var kubeconfig *string

func init() {
    if home, _ := os.UserHomeDir(); home != "" {
        kubeconfig = flag.String("kubeconfig", filepath.Join(home, ".kube",
```

```
"config"), "path to kubeconfig")
    } else {
        kubeconfig = flag.String("kubeconfig", "", "path to kubeconfig")
    }
    flag.Parse()
}

config, err := clientcmd.BuildConfigFromFlags("", *kubeconfig)
```

Method 2: In-Cluster Config (Pod Inside Kubernetes)

go

```
import "k8s.io/client-go/rest"

func newInClusterClient() (*kubernetes.Clientset, error) {
    // Reads token + CA from /var/run/secrets/kubernetes.io/serviceaccount/
    config, err := rest.InClusterConfig()
    if err != nil {
        return nil, err
    }
    return kubernetes.NewForConfig(config)
}
```

Method 3: Auto-detect (In-cluster or kubeconfig)

The most common production pattern — works both locally and in-cluster:

go

```
func buildClient() (*kubernetes.Clientset, error) {
    // Try in-cluster first
    config, err := rest.InClusterConfig()
    if err != nil {
        // Fall back to kubeconfig
        home, _ := os.UserHomeDir()
        kubeconfig := filepath.Join(home, ".kube", "config")
        config, err = clientcmd.BuildConfigFromFlags("", kubeconfig)
        if err != nil {
            return nil, fmt.Errorf("cannot build config: %w", err)
        }
```

```
    }
    return kubernetes.NewForConfig(config)
}
```

Method 4: Specific Context from kubeconfig

```go
func clientForContext(kubeconfigPath, context string) (*kubernetes.Clientset, error) {
    loadingRules := clientcmd.NewDefaultClientConfigLoadingRules()
    loadingRules.ExplicitPath = kubeconfigPath

    configOverrides := &clientcmd.ConfigOverrides{
        CurrentContext: context,
    }

    config, err := clientcmd.NewNonInteractiveDeferredLoadingClientConfig(
        loadingRules, configOverrides,
    ).ClientConfig()
    if err != nil {
        return nil, err
    }

    return kubernetes.NewForConfig(config)
}

// Usage
cs, err := clientForContext("~/.kube/config", "prod-cluster")
```

Method 5: Manual REST Config (Token / Cert)

```go
// Bearer token
config := &rest.Config{
    Host:        "https://k8s.example.com:6443",
    BearerToken: "eyJhbGci...",
    TLSClientConfig: rest.TLSClientConfig{
        CAFile: "/path/to/ca.crt",
    },
```

```
}

// Client certificate
config := &rest.Config{
    Host: "https://k8s.example.com:6443",
    TLSClientConfig: rest.TLSClientConfig{
        CAFile:   "/path/to/ca.crt",
        CertFile: "/path/to/client.crt",
        KeyFile:  "/path/to/client.key",
    },
}

clientset, err := kubernetes.NewForConfig(config)
```

Multi-Cluster Management

go

```
type MultiClusterManager struct {
    clients map[string]*kubernetes.Clientset
}

func NewMultiClusterManager(contexts map[string]string) (*MultiClusterManager,
error) {
    m := &MultiClusterManager{clients: make(map[string]*kubernetes.Clientset)}

    for name, ctx := range contexts {
        cs, err := clientForContext("~/.kube/config", ctx)
        if err != nil {
            return nil, fmt.Errorf("cluster %s: %w", name, err)
        }
        m.clients[name] = cs
    }
    return m, nil
}

func (m *MultiClusterManager) For(cluster string) *kubernetes.Clientset {
    return m.clients[cluster]
}

// Usage
mgr, _ := NewMultiClusterManager(map[string]string{
    "us-east": "eks-us-east-1",
    "eu-west": "gke-europe-west1",
```

```
})

pods, _ := mgr.For("us-east").CoreV1().Pods("default").List(ctx,
metav1.ListOptions{})
```

Tuning the REST Config

```go
config, _ := clientcmd.BuildConfigFromFlags("", kubeconfig)

// Increase QPS and Burst for high-throughput controllers
config.QPS   = 100    // default: 5 requests/sec
config.Burst = 200    // default: 10 burst

// Timeout per request
config.Timeout = 30 * time.Second

clientset, _ := kubernetes.NewForConfig(config)
```

Verifying the Connection

```go
func verifyConnection(clientset *kubernetes.Clientset) error {
    version, err := clientset.Discovery().ServerVersion()
    if err != nil {
        return fmt.Errorf("cannot reach API server: %w", err)
    }
    fmt.Printf("Connected to Kubernetes %s\n", version.GitVersion)
    return nil
}
```

Connection Troubleshooting

Error	Cause	Fix
`connection refused`	Wrong host/port	Check `kubectl cluster-info`
`x509: certificate signed by unknown authority`	Wrong CA	Verify `CAFile` or `InsecureSkipVerify` (dev only)
`Unauthorized (401)`	Bad/expired token	Regenerate token
`Forbidden (403)`	No RBAC permission	Grant Role to ServiceAccount
`dial tcp: no route to host`	Firewall / VPN	Check network access

```go
// Dev only — skip TLS verification
config.TLSClientConfig = rest.TLSClientConfig{Insecure: true}
```

Chapter 4: Working with Pods

Pod Interface

```go
clientset.CoreV1().Pods(namespace)
```

All methods accept a `context.Context` as the first argument — always pass one with a timeout.

Listing Pods

```go
import (
    "context"
    "fmt"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

ctx := context.TODO()

// All pods in a namespace
```

```
pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{})
if err != nil {
    log.Fatalf("Error listing pods: %v", err)
}

for _, pod := range pods.Items {
    fmt.Printf("%-40s %-10s %s\n", pod.Name, pod.Status.Phase,
pod.Spec.NodeName)
}
```

Filter by Label Selector

```go
pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{
    LabelSelector: "app=nginx,env=production",
})

// Programmatic label selector
import "k8s.io/apimachinery/pkg/labels"

selector := labels.Set{"app": "nginx", "env": "production"}.String()
pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{
    LabelSelector: selector,
})
```

Filter by Field Selector

```go
pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{
    FieldSelector: "status.phase=Running",
})

// Multiple field selectors
pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{
    FieldSelector: "status.phase=Running,spec.nodeName=node-1",
})
```

All Namespaces

```go
pods, err := clientset.CoreV1().Pods("").List(ctx, metav1.ListOptions{})
// "" = all namespaces (like kubectl get pods -A)
```

Getting a Single Pod

```go
pod, err := clientset.CoreV1().Pods("default").Get(ctx, "my-nginx",
metav1.GetOptions{})
if err != nil {
    if errors.IsNotFound(err) {
        fmt.Println("Pod not found")
        return
    }
    log.Fatalf("Error getting pod: %v", err)
}

fmt.Printf("Name:      %s\n", pod.Name)
fmt.Printf("Namespace: %s\n", pod.Namespace)
fmt.Printf("Phase:     %s\n", pod.Status.Phase)
fmt.Printf("Node:      %s\n", pod.Spec.NodeName)
fmt.Printf("PodIP:     %s\n", pod.Status.PodIP)
```

Creating a Pod

```go
import (
    corev1 "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/api/resource"
)

pod := &corev1.Pod{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-nginx",
        Namespace: "default",
        Labels:    map[string]string{"app": "nginx", "env": "dev"},
    },
    Spec: corev1.PodSpec{
        Containers: []corev1.Container{
            {
                Name:  "nginx",
                Image: "nginx:1.25-alpine",
                Ports: []corev1.ContainerPort{
                    {ContainerPort: 80},
                },
                Resources: corev1.ResourceRequirements{
```

```
                    Requests: corev1.ResourceList{
                        corev1.ResourceCPU:    resource.MustParse("100m"),
                        corev1.ResourceMemory: resource.MustParse("64Mi"),
                    },
                    Limits: corev1.ResourceList{
                        corev1.ResourceCPU:    resource.MustParse("200m"),
                        corev1.ResourceMemory: resource.MustParse("128Mi"),
                    },
                },
                Env: []corev1.EnvVar{
                    {Name: "ENV", Value: "production"},
                },
            },
        },
        RestartPolicy: corev1.RestartPolicyAlways,
    },
}

created, err := clientset.CoreV1().Pods("default").Create(ctx, pod,
metav1.CreateOptions{})
if err != nil {
    log.Fatalf("Error creating pod: %v", err)
}
fmt.Printf("Pod created: %s\n", created.Name)
```

Pod Logs

go

```
import (
    "io"
    corev1 "k8s.io/api/core/v1"
)

// Fetch logs
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{})
stream, err := req.Stream(ctx)
if err != nil {
    log.Fatalf("Error getting logs: %v", err)
}
defer stream.Close()
```

```
logs, _ := io.ReadAll(stream)
fmt.Println(string(logs))
```

Log Options

go

```
tailLines := int64(100)
sinceSeconds := int64(3600)

// Last 100 lines
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{
    TailLines: &tailLines,
})

// Last hour
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{
    SinceSeconds: &sinceSeconds,
})

// Previous crashed container
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{
    Previous: true,
})

// Specific container (multi-container pod)
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{
    Container: "sidecar",
})

// Follow (streaming)
req := clientset.CoreV1().Pods("default").GetLogs("my-nginx",
&corev1.PodLogOptions{
    Follow: true,
})
stream, _ := req.Stream(ctx)
io.Copy(os.Stdout, stream)
```

Deleting a Pod

```go
// Delete by name
err := clientset.CoreV1().Pods("default").Delete(ctx, "my-nginx",
metav1.DeleteOptions{})

// Force delete (grace period = 0)
gracePeriod := int64(0)
err := clientset.CoreV1().Pods("default").Delete(ctx, "my-nginx",
metav1.DeleteOptions{
    GracePeriodSeconds: &gracePeriod,
})
```

Exec into a Pod

```go
import (
    "k8s.io/client-go/tools/remotecommand"
    "k8s.io/client-go/kubernetes/scheme"
)

func execInPod(clientset *kubernetes.Clientset, config *rest.Config, namespace,
podName, container string, command []string) (string, string, error) {
    req := clientset.CoreV1().RESTClient().Post().
        Resource("pods").
        Name(podName).
        Namespace(namespace).
        SubResource("exec").
        VersionedParams(&corev1.PodExecOptions{
            Container: container,
            Command:   command,
            Stdin:     false,
            Stdout:    true,
            Stderr:    true,
            TTY:       false,
        }, scheme.ParameterCodec)

    exec, err := remotecommand.NewSPDYExecutor(config, "POST", req.URL())
    if err != nil {
        return "", "", err
    }
```

```
    var stdout, stderr bytes.Buffer
    err = exec.StreamWithContext(ctx, remotecommand.StreamOptions{
        Stdout: &stdout,
        Stderr: &stderr,
    })

    return stdout.String(), stderr.String(), err
}

// Usage
stdout, stderr, err := execInPod(clientset, config, "default", "my-pod", "app",
[]string{"ls", "/var/data"})
```

Watch Pods

go

```
watcher, err := clientset.CoreV1().Pods("default").Watch(ctx,
metav1.ListOptions{})
if err != nil {
    log.Fatal(err)
}
defer watcher.Stop()

for event := range watcher.ResultChan() {
    pod, ok := event.Object.(*corev1.Pod)
    if !ok {
        continue
    }
    fmt.Printf("[%s] Pod: %s Phase: %s\n", event.Type, pod.Name,
pod.Status.Phase)
}
```

Wait for Pod Running

go

```
import (
    "k8s.io/apimachinery/pkg/watch"
    "k8s.io/apimachinery/pkg/fields"
)
```

```
func waitForPodRunning(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string) error {
    watcher, err := clientset.CoreV1().Pods(namespace).Watch(ctx,
metav1.ListOptions{
        FieldSelector: fields.OneTermEqualSelector("metadata.name",
name).String(),
    })
    if err != nil {
        return err
    }
    defer watcher.Stop()

    for event := range watcher.ResultChan() {
        pod, ok := event.Object.(*corev1.Pod)
        if !ok {
            continue
        }
        if pod.Status.Phase == corev1.PodRunning {
            return nil
        }
        if pod.Status.Phase == corev1.PodFailed {
            return fmt.Errorf("pod %s failed", name)
        }
    }
    return fmt.Errorf("watch channel closed")
}
```

Container Status Check

go

```
pod, _ := clientset.CoreV1().Pods("default").Get(ctx, "my-pod",
metav1.GetOptions{})

for _, cs := range pod.Status.ContainerStatuses {
    fmt.Printf("Container: %-20s Ready: %-5v Restarts: %d\n",
        cs.Name, cs.Ready, cs.RestartCount)

    if cs.State.Waiting != nil {
        fmt.Printf("  Waiting: %s — %s\n", cs.State.Waiting.Reason,
cs.State.Waiting.Message)
    }
```

```
    if cs.State.Terminated != nil {
        fmt.Printf("  Exit code: %d\n", cs.State.Terminated.ExitCode)
    }
}
```

Chapter 5: Deployments & ReplicaSets

Deployment Interface

```go
clientset.AppsV1().Deployments(namespace)
```

Listing Deployments

```go
deployments, err := clientset.AppsV1().Deployments("default").List(ctx,
metav1.ListOptions{})
if err != nil {
    log.Fatal(err)
}

for _, d := range deployments.Items {
    fmt.Printf("%-40s  %d/%d ready\n",
        d.Name,
        d.Status.ReadyReplicas,
        *d.Spec.Replicas,
    )
}
```

Getting a Single Deployment

```go
dep, err := clientset.AppsV1().Deployments("default").Get(ctx, "my-app",
metav1.GetOptions{})
if err != nil {
    log.Fatal(err)
}
```

```
fmt.Printf("Desired:   %d\n", *dep.Spec.Replicas)
fmt.Printf("Ready:     %d\n", dep.Status.ReadyReplicas)
fmt.Printf("Updated:   %d\n", dep.Status.UpdatedReplicas)
fmt.Printf("Available: %d\n", dep.Status.AvailableReplicas)
fmt.Printf("Image:     %s\n", dep.Spec.Template.Spec.Containers[0].Image)
```

Creating a Deployment

go

```
import (
    appsv1  "k8s.io/api/apps/v1"
    corev1  "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/api/resource"
    "k8s.io/apimachinery/pkg/util/intstr"
)

func int32Ptr(i int32) *int32 { return &i }

deployment := &appsv1.Deployment{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-app",
        Namespace: "default",
        Labels:    map[string]string{"app": "my-app"},
    },
    Spec: appsv1.DeploymentSpec{
        Replicas: int32Ptr(3),
        Selector: &metav1.LabelSelector{
            MatchLabels: map[string]string{"app": "my-app"},
        },
        Strategy: appsv1.DeploymentStrategy{
            Type: appsv1.RollingUpdateDeploymentStrategyType,
            RollingUpdate: &appsv1.RollingUpdateDeployment{
                MaxUnavailable: &intstr.IntOrString{Type: intstr.Int, IntVal:
1},
                MaxSurge:       &intstr.IntOrString{Type: intstr.Int, IntVal:
1},
            },
        },
        Template: corev1.PodTemplateSpec{
            ObjectMeta: metav1.ObjectMeta{
                Labels: map[string]string{"app": "my-app"},
            },
```

```
            Spec: corev1.PodSpec{
                Containers: []corev1.Container{
                    {
                        Name:  "my-app",
                        Image: "myregistry/my-app:1.0.0",
                        Ports: []corev1.ContainerPort{{ContainerPort: 8080}},
                        Resources: corev1.ResourceRequirements{
                            Requests: corev1.ResourceList{
                                corev1.ResourceCPU:
resource.MustParse("100m"),
                                corev1.ResourceMemory:
resource.MustParse("128Mi"),
                            },
                            Limits: corev1.ResourceList{
                                corev1.ResourceCPU:
resource.MustParse("500m"),
                                corev1.ResourceMemory:
resource.MustParse("256Mi"),
                            },
                        },
                        ReadinessProbe: &corev1.Probe{
                            ProbeHandler: corev1.ProbeHandler{
                                HTTPGet: &corev1.HTTPGetAction{
                                    Path: "/health",
                                    Port: intstr.FromInt(8080),
                                },
                            },
                            InitialDelaySeconds: 5,
                            PeriodSeconds:       10,
                        },
                    },
                },
            },
        },
    },
}

created, err := clientset.AppsV1().Deployments("default").Create(ctx,
deployment, metav1.CreateOptions{})
if err != nil {
    log.Fatalf("Error creating deployment: %v", err)
}
fmt.Printf("Deployment created: %s\n", created.Name)
```

Scaling a Deployment

```go
import "k8s.io/client-go/util/retry"

func scaleDeployment(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string, replicas int32) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := clientset.AppsV1().Deployments(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        dep.Spec.Replicas = &replicas
        _, err = clientset.AppsV1().Deployments(namespace).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
}

scaleDeployment(ctx, clientset, "default", "my-app", 5)
```

Update Image (Rolling Update)

```go
func updateDeploymentImage(ctx context.Context, clientset
*kubernetes.Clientset, namespace, name, container, image string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := clientset.AppsV1().Deployments(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }

        for i, c := range dep.Spec.Template.Spec.Containers {
            if c.Name == container {
                dep.Spec.Template.Spec.Containers[i].Image = image
                break
            }
        }

        _, err = clientset.AppsV1().Deployments(namespace).Update(ctx, dep,
```

```
metav1.UpdateOptions{})
        return err
    })
}

updateDeploymentImage(ctx, clientset, "default", "my-app", "my-app",
"myregistry/my-app:2.0.0")
```

Patch a Deployment (Strategic Merge Patch)

go

```
import (
    "encoding/json"
    "k8s.io/apimachinery/pkg/types"
)

type patchReplicas struct {
    Spec struct {
        Replicas int32 `json:"replicas"`
    } `json:"spec"`
}

patch := patchReplicas{}
patch.Spec.Replicas = 5

data, _ := json.Marshal(patch)
_, err = clientset.AppsV1().Deployments("default").Patch(
    ctx, "my-app",
    types.MergePatchType,
    data,
    metav1.PatchOptions{},
)
```

Wait for Rollout Completion

go

```
func waitForDeploymentRollout(ctx context.Context, clientset
*kubernetes.Clientset, namespace, name string) error {
    for {
```

```
        dep, err := clientset.AppsV1().Deployments(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }

        desired   := *dep.Spec.Replicas
        updated   := dep.Status.UpdatedReplicas
        ready     := dep.Status.ReadyReplicas
        available := dep.Status.AvailableReplicas

        fmt.Printf("updated=%d ready=%d available=%d desired=%d\n",
            updated, ready, available, desired)

        if updated == desired && ready == desired && available == desired {
            fmt.Println("Rollout complete!")
            return nil
        }

        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
        }
    }
}

timeoutCtx, cancel := context.WithTimeout(context.Background(), 5*time.Minute)
defer cancel()
waitForDeploymentRollout(timeoutCtx, clientset, "default", "my-app")
```

Rollout Restart (Force Pod Recreation)

go

```
func rolloutRestart(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := clientset.AppsV1().Deployments(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
```

```
        if dep.Spec.Template.Annotations == nil {
            dep.Spec.Template.Annotations = map[string]string{}
        }
        dep.Spec.Template.Annotations["kubectl.kubernetes.io/restartedAt"] =
time.Now().Format(time.RFC3339)

        _, err = clientset.AppsV1().Deployments(namespace).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
}
```

Delete a Deployment

go

```
// Delete deployment and all its pods (propagation)
propagation := metav1.DeletePropagationForeground
err := clientset.AppsV1().Deployments("default").Delete(ctx, "my-app",
metav1.DeleteOptions{
    PropagationPolicy: &propagation,
})
```

Working with ReplicaSets

go

```
// List ReplicaSets for a deployment
rsets, err := clientset.AppsV1().ReplicaSets("default").List(ctx,
metav1.ListOptions{
    LabelSelector: "app=my-app",
})

for _, rs := range rsets.Items {
    fmt.Printf("%-40s  desired=%d ready=%d\n",
        rs.Name, *rs.Spec.Replicas, rs.Status.ReadyReplicas)
}
```

Deployment Status Conditions

```go
dep, _ := clientset.AppsV1().Deployments("default").Get(ctx, "my-app",
metav1.GetOptions{})

for _, cond := range dep.Status.Conditions {
    fmt.Printf("%-25s %-8s %s\n", cond.Type, cond.Status, cond.Message)
}
// Types: Available, Progressing, ReplicaFailure
```

Chapter 6: Services & Networking

Service Interface

```go
clientset.CoreV1().Services(namespace)
```

Listing Services

```go
services, err := clientset.CoreV1().Services("default").List(ctx,
metav1.ListOptions{})
if err != nil {
    log.Fatal(err)
}

for _, svc := range services.Items {
    ports := make([]string, len(svc.Spec.Ports))
    for i, p := range svc.Spec.Ports {
        ports[i] = fmt.Sprintf("%d/%s", p.Port, p.Protocol)
    }
    fmt.Printf("%-35s %-15s %-16s %s\n",
        svc.Name, svc.Spec.Type, svc.Spec.ClusterIP, strings.Join(ports, ","))
}
```

Creating a ClusterIP Service

```go
import (
    corev1 "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/util/intstr"
)

svc := &corev1.Service{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-app-svc",
        Namespace: "default",
        Labels:    map[string]string{"app": "my-app"},
    },
    Spec: corev1.ServiceSpec{
        Type:     corev1.ServiceTypeClusterIP,
        Selector: map[string]string{"app": "my-app"},
        Ports: []corev1.ServicePort{
            {
                Name:       "http",
                Port:       80,
                TargetPort: intstr.FromInt(8080),
                Protocol:   corev1.ProtocolTCP,
            },
        },
    },
}

created, err := clientset.CoreV1().Services("default").Create(ctx, svc,
metav1.CreateOptions{})
fmt.Printf("Service created: %s (%s)\n", created.Name, created.Spec.ClusterIP)
```

Creating a NodePort Service

```go
nodePort := int32(30080)

svc := &corev1.Service{
    ObjectMeta: metav1.ObjectMeta{Name: "my-app-nodeport", Namespace:
"default"},
    Spec: corev1.ServiceSpec{
        Type:     corev1.ServiceTypeNodePort,
        Selector: map[string]string{"app": "my-app"},
        Ports: []corev1.ServicePort{
            {
```

```
                Port:       80,
                TargetPort: intstr.FromInt(8080),
                NodePort:   nodePort,
            },
        },
    },
}

clientset.CoreV1().Services("default").Create(ctx, svc, metav1.CreateOptions{})
```

Creating a LoadBalancer Service

```go
svc := &corev1.Service{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-app-lb",
        Namespace: "default",
        Annotations: map[string]string{
            "service.beta.kubernetes.io/aws-load-balancer-type": "nlb",
        },
    },
    Spec: corev1.ServiceSpec{
        Type:     corev1.ServiceTypeLoadBalancer,
        Selector: map[string]string{"app": "my-app"},
        Ports: []corev1.ServicePort{
            {Port: 80,  TargetPort: intstr.FromInt(8080)},
            {Port: 443, TargetPort: intstr.FromInt(8443)},
        },
    },
}

clientset.CoreV1().Services("default").Create(ctx, svc, metav1.CreateOptions{})
```

Wait for LoadBalancer IP

```go
func waitForLoadBalancerIP(ctx context.Context, clientset
*kubernetes.Clientset, namespace, name string) (string, error) {
    for {
        svc, err := clientset.CoreV1().Services(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
```

```
            return "", err
        }

        if len(svc.Status.LoadBalancer.Ingress) > 0 {
            ingress := svc.Status.LoadBalancer.Ingress[0]
            if ingress.IP != "" {
                return ingress.IP, nil
            }
            if ingress.Hostname != "" {
                return ingress.Hostname, nil
            }
        }

        select {
        case <-ctx.Done():
            return "", ctx.Err()
        case <-time.After(5 * time.Second):
            fmt.Println("Waiting for external IP...")
        }
    }
}
```

Creating a Headless Service

go

```
svc := &corev1.Service{
    ObjectMeta: metav1.ObjectMeta{Name: "mysql-headless"},
    Spec: corev1.ServiceSpec{
        ClusterIP: "None",   // headless
        Selector:  map[string]string{"app": "mysql"},
        Ports: []corev1.ServicePort{
            {Port: 3306, Name: "mysql"},
        },
    },
}
```

Creating an ExternalName Service

```go
svc := &corev1.Service{
    ObjectMeta: metav1.ObjectMeta{Name: "external-db"},
    Spec: corev1.ServiceSpec{
        Type:         corev1.ServiceTypeExternalName,
        ExternalName: "mydb.rds.amazonaws.com",
    },
}
```

Getting Service Details

```go
svc, err := clientset.CoreV1().Services("default").Get(ctx, "my-app-svc",
metav1.GetOptions{})

fmt.Printf("ClusterIP: %s\n", svc.Spec.ClusterIP)
fmt.Printf("Type:      %s\n", svc.Spec.Type)

for _, p := range svc.Spec.Ports {
    fmt.Printf("Port: %d → %s\n", p.Port, p.TargetPort.String())
}

// External IPs for LoadBalancer
for _, ing := range svc.Status.LoadBalancer.Ingress {
    fmt.Printf("External: %s%s\n", ing.IP, ing.Hostname)
}
```

Working with Endpoints

```go
ep, err := clientset.CoreV1().Endpoints("default").Get(ctx, "my-app-svc",
metav1.GetOptions{})

for _, subset := range ep.Subsets {
    port := subset.Ports[0].Port
    for _, addr := range subset.Addresses {
        fmt.Printf("Ready:    %s:%d (pod=%s)\n", addr.IP, port,
addr.TargetRef.Name)
    }
    for _, addr := range subset.NotReadyAddresses {
```

```
            fmt.Printf("NotReady: %s:%d\n", addr.IP, port)
        }
    }
```

Update a Service

```go
import "k8s.io/client-go/util/retry"

err = retry.RetryOnConflict(retry.DefaultRetry, func() error {
    svc, err := clientset.CoreV1().Services("default").Get(ctx, "my-app-svc",
metav1.GetOptions{})
    if err != nil {
        return err
    }
    // Add annotation
    svc.Annotations["custom.io/updated"] = time.Now().Format(time.RFC3339)
    _, err = clientset.CoreV1().Services("default").Update(ctx, svc,
metav1.UpdateOptions{})
    return err
})
```

Delete a Service

```go
err := clientset.CoreV1().Services("default").Delete(ctx, "my-app-svc",
metav1.DeleteOptions{})
```

NetworkPolicy (Read)

```go
netPolicies, _ := clientset.NetworkingV1().NetworkPolicies("default").List(ctx,
metav1.ListOptions{})

for _, np := range netPolicies.Items {
    fmt.Printf("Policy: %s → podSelector: %v\n",
```

```
        np.Name, np.Spec.PodSelector.MatchLabels)
}
```

Chapter 7: ConfigMaps & Secrets

ConfigMap Interface

```go
clientset.CoreV1().ConfigMaps(namespace)
clientset.CoreV1().Secrets(namespace)
```

Working with ConfigMaps

List ConfigMaps

```go
cms, err := clientset.CoreV1().ConfigMaps("default").List(ctx,
metav1.ListOptions{})
for _, cm := range cms.Items {
    keys := make([]string, 0, len(cm.Data))
    for k := range cm.Data {
        keys = append(keys, k)
    }
    fmt.Printf("%-30s keys: %s\n", cm.Name, strings.Join(keys, ", "))
}
```

Get a ConfigMap

```go
cm, err := clientset.CoreV1().ConfigMaps("default").Get(ctx, "app-config",
metav1.GetOptions{})
if err != nil {
    if errors.IsNotFound(err) {
        fmt.Println("ConfigMap not found")
        return
    }
    log.Fatal(err)
}
```

```
fmt.Println("DB_HOST:", cm.Data["DB_HOST"])
fmt.Println("APP_ENV:", cm.Data["APP_ENV"])
```

Create a ConfigMap

```go
import corev1 "k8s.io/api/core/v1"

cm := &corev1.ConfigMap{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "app-config",
        Namespace: "default",
        Labels:    map[string]string{"app": "my-app"},
    },
    Data: map[string]string{
        "APP_ENV":   "production",
        "DB_HOST":   "postgres.default.svc.cluster.local",
        "DB_PORT":   "5432",
        "CACHE_TTL": "3600",
        "nginx.conf": `server {
    listen 80;
    server_name _;
    location / { proxy_pass http://app:8080; }
}`,
    },
}

created, err := clientset.CoreV1().ConfigMaps("default").Create(ctx, cm,
metav1.CreateOptions{})
fmt.Printf("ConfigMap created: %s\n", created.Name)
```

Update a ConfigMap

```go
import "k8s.io/client-go/util/retry"

err = retry.RetryOnConflict(retry.DefaultRetry, func() error {
    cm, err := clientset.CoreV1().ConfigMaps("default").Get(ctx, "app-config",
metav1.GetOptions{})
    if err != nil {
        return err
    }
    cm.Data["CACHE_TTL"] = "7200"
    cm.Data["NEW_FLAG"]  = "true"
    _, err = clientset.CoreV1().ConfigMaps("default").Update(ctx, cm,
```

```
metav1.UpdateOptions{})
    return err
})
```

Apply (Create or Update)

```go
func applyConfigMap(ctx context.Context, clientset *kubernetes.Clientset,
namespace string, data map[string]string) error {
    name := "app-config"
    cm := &corev1.ConfigMap{
        ObjectMeta: metav1.ObjectMeta{Name: name, Namespace: namespace},
        Data:       data,
    }

    _, err := clientset.CoreV1().ConfigMaps(namespace).Get(ctx, name,
metav1.GetOptions{})
    if errors.IsNotFound(err) {
        _, err = clientset.CoreV1().ConfigMaps(namespace).Create(ctx, cm,
metav1.CreateOptions{})
        return err
    }

    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        existing, err := clientset.CoreV1().ConfigMaps(namespace).Get(ctx,
name, metav1.GetOptions{})
        if err != nil {
            return err
        }
        existing.Data = data
        _, err = clientset.CoreV1().ConfigMaps(namespace).Update(ctx, existing,
metav1.UpdateOptions{})
        return err
    })
}
```

Delete a ConfigMap

```go
err := clientset.CoreV1().ConfigMaps("default").Delete(ctx, "app-config",
metav1.DeleteOptions{})
```

Working with Secrets

> **Security Rule:** Never log secret values. Handle them only in memory when needed.

List Secrets

```go
secrets, err := clientset.CoreV1().Secrets("default").List(ctx,
metav1.ListOptions{})
for _, s := range secrets.Items {
    keys := make([]string, 0, len(s.Data))
    for k := range s.Data {
        keys = append(keys, k)
    }
    fmt.Printf("%-30s type=%-40s keys=%s\n", s.Name, s.Type, strings.Join(keys,
","))
}
```

Get and Decode a Secret

```go
secret, err := clientset.CoreV1().Secrets("default").Get(ctx, "db-credentials",
metav1.GetOptions{})
if err != nil {
    log.Fatal(err)
}

// Data is already []byte — no need to base64 decode (client-go does it)
username := string(secret.Data["username"])
password := string(secret.Data["password"])

dsn := fmt.Sprintf("postgres://%s:%s@postgres:5432/mydb", username, password)
```

Create an Opaque Secret

```go
secret := &corev1.Secret{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "db-credentials",
        Namespace: "default",
        Labels:    map[string]string{"app": "my-app"},
    },
    Type: corev1.SecretTypeOpaque,
    // StringData is auto base64-encoded by the API
```

```
    StringData: map[string]string{
        "username": "admin",
        "password": "super-secret-password",
        "db_url":   "postgres://admin:pass@postgres:5432/mydb",
    },
}

clientset.CoreV1().Secrets("default").Create(ctx, secret,
metav1.CreateOptions{})
```

Create a Docker Registry Secret

go

```
import "encoding/json"

dockerConfig := map[string]interface{}{
    "auths": map[string]interface{}{
        "registry.example.com": map[string]string{
            "username": "myuser",
            "password": "mypass",
            "email":    "ci@example.com",
            "auth":
base64.StdEncoding.EncodeToString([]byte("myuser:mypass")),
        },
    },
}

dockerConfigJSON, _ := json.Marshal(dockerConfig)

secret := &corev1.Secret{
    ObjectMeta: metav1.ObjectMeta{Name: "registry-credentials", Namespace:
"default"},
    Type:       corev1.SecretTypeDockerConfigJson,
    Data: map[string][]byte{
        corev1.DockerConfigJsonKey: dockerConfigJSON,
    },
}

clientset.CoreV1().Secrets("default").Create(ctx, secret,
metav1.CreateOptions{})
```

Create a TLS Secret

```go
secret := &corev1.Secret{
    ObjectMeta: metav1.ObjectMeta{Name: "my-tls-secret", Namespace: "default"},
    Type:       corev1.SecretTypeTLS,
    Data: map[string][]byte{
        corev1.TLSCertKey:       certPEM,   // []byte of the cert
        corev1.TLSPrivateKeyKey: keyPEM,    // []byte of the key
    },
}

clientset.CoreV1().Secrets("default").Create(ctx, secret,
metav1.CreateOptions{})
```

Update a Secret (Rotate Password)

```go
func rotateSecret(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name, key, newValue string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        secret, err := clientset.CoreV1().Secrets(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }

        // Keep previous value for rollback
        if val, ok := secret.Data[key]; ok {
            secret.Data[key+"_previous"] = val
        }
        secret.Data[key] = []byte(newValue)

        _, err = clientset.CoreV1().Secrets(namespace).Update(ctx, secret,
metav1.UpdateOptions{})
        return err
    })
}
```

Delete a Secret

```go
err := clientset.CoreV1().Secrets("default").Delete(ctx, "db-credentials",
metav1.DeleteOptions{})
```

Chapter 8: Persistent Volumes & Storage

Storage Interfaces

```go
clientset.CoreV1().PersistentVolumes()                    // cluster-scoped
clientset.CoreV1().PersistentVolumeClaims(namespace)      // namespaced
clientset.StorageV1().StorageClasses()                    // cluster-scoped
```

Working with StorageClasses

List StorageClasses

```go
scs, err := clientset.StorageV1().StorageClasses().List(ctx,
metav1.ListOptions{})
for _, sc := range scs.Items {
    isDefault := sc.Annotations["storageclass.kubernetes.io/is-default-class"]
== "true"
    defaultStr := ""
    if isDefault {
        defaultStr = "(default)"
    }
    fmt.Printf("%-30s %-40s %-10s %s\n",
        sc.Name, sc.Provisioner, *sc.ReclaimPolicy, defaultStr)
}
```

Create a StorageClass

```go
import (
    storagev1 "k8s.io/api/storage/v1"
    corev1    "k8s.io/api/core/v1"
)

reclaimPolicy    := corev1.PersistentVolumeReclaimRetain
bindingMode      := storagev1.VolumeBindingWaitForFirstConsumer

sc := &storagev1.StorageClass{
    ObjectMeta: metav1.ObjectMeta{
        Name: "fast-ssd",
        Annotations: map[string]string{
            "storageclass.kubernetes.io/is-default-class": "false",
        },
```

```
    },
    Provisioner:          "kubernetes.io/gce-pd",
    ReclaimPolicy:        &reclaimPolicy,
    VolumeBindingMode:    &bindingMode,
    Parameters: map[string]string{
        "type":             "pd-ssd",
        "replication-type": "regional-pd",
    },
}

clientset.StorageV1().StorageClasses().Create(ctx, sc, metav1.CreateOptions{})
```

Working with PersistentVolumeClaims

List PVCs

```go
pvcs, err := clientset.CoreV1().PersistentVolumeClaims("default").List(ctx,
metav1.ListOptions{})
for _, pvc := range pvcs.Items {
    capacity := pvc.Status.Capacity.Storage()
    fmt.Printf("%-30s %-10s %-10s %-20s %s\n",
        pvc.Name,
        pvc.Status.Phase,
        capacity.String(),
        pvc.Spec.StorageClassName,
        strings.Join(pvc.Spec.AccessModes, ","),
    )
}
```

Create a PVC

```go
import (
    corev1 "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/api/resource"
)

storageClass := "standard"

pvc := &corev1.PersistentVolumeClaim{
    ObjectMeta: metav1.ObjectMeta{
```

```
        Name:      "my-data-pvc",
        Namespace: "default",
        Labels:    map[string]string{"app": "my-app"},
    },
    Spec: corev1.PersistentVolumeClaimSpec{
        AccessModes:
[]corev1.PersistentVolumeAccessMode{corev1.ReadWriteOnce},
        StorageClassName: &storageClass,
        Resources: corev1.ResourceRequirements{
            Requests: corev1.ResourceList{
                corev1.ResourceStorage: resource.MustParse("10Gi"),
            },
        },
    },
}

created, err :=
clientset.CoreV1().PersistentVolumeClaims("default").Create(ctx, pvc,
metav1.CreateOptions{})
fmt.Printf("PVC created: %s\n", created.Name)
```

Wait for PVC to be Bound

go

```
func waitForPVCBound(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string) error {
    for {
        pvc, err :=
clientset.CoreV1().PersistentVolumeClaims(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }

        if pvc.Status.Phase == corev1.ClaimBound {
            fmt.Printf("PVC %s bound to PV %s\n", name, pvc.Spec.VolumeName)
            return nil
        }

        fmt.Printf("PVC phase: %s — waiting...\n", pvc.Status.Phase)
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
        }
```

```
    }
}
```

PVC with ReadWriteMany (Shared Storage)

```go
storageClass := "nfs-client"

pvc := &corev1.PersistentVolumeClaim{
    ObjectMeta: metav1.ObjectMeta{Name: "shared-data", Namespace: "default"},
    Spec: corev1.PersistentVolumeClaimSpec{
        AccessModes:
[]corev1.PersistentVolumeAccessMode{corev1.ReadWriteMany},
        StorageClassName: &storageClass,
        Resources: corev1.ResourceRequirements{
            Requests: corev1.ResourceList{
                corev1.ResourceStorage: resource.MustParse("50Gi"),
            },
        },
    },
}
```

Working with PersistentVolumes

List PVs

```go
pvs, err := clientset.CoreV1().PersistentVolumes().List(ctx,
metav1.ListOptions{})
for _, pv := range pvs.Items {
    capacity := pv.Spec.Capacity.Storage()
    claim := ""
    if pv.Spec.ClaimRef != nil {
        claim = pv.Spec.ClaimRef.Namespace + "/" + pv.Spec.ClaimRef.Name
    }
    fmt.Printf("%-30s %-8s %-10s %-10s %s\n",
        pv.Name, capacity.String(), pv.Status.Phase,
pv.Spec.PersistentVolumeReclaimPolicy, claim)
}
```

Create a PV (NFS Static Provisioning)

```go
pv := &corev1.PersistentVolume{
    ObjectMeta: metav1.ObjectMeta{
        Name:   "my-nfs-pv",
        Labels: map[string]string{"type": "nfs"},
    },
    Spec: corev1.PersistentVolumeSpec{
        Capacity: corev1.ResourceList{
            corev1.ResourceStorage: resource.MustParse("100Gi"),
        },
        AccessModes:
[]corev1.PersistentVolumeAccessMode{corev1.ReadWriteMany},
        PersistentVolumeReclaimPolicy: corev1.PersistentVolumeReclaimRetain,
        StorageClassName:              "nfs",
        PersistentVolumeSource: corev1.PersistentVolumeSource{
            NFS: &corev1.NFSVolumeSource{
                Server: "10.0.0.5",
                Path:   "/exported/data",
            },
        },
    },
}

clientset.CoreV1().PersistentVolumes().Create(ctx, pv, metav1.CreateOptions{})
```

Mount PVC in a Deployment

```go
deployment := &appsv1.Deployment{
    ObjectMeta: metav1.ObjectMeta{Name: "app-with-storage", Namespace:
"default"},
    Spec: appsv1.DeploymentSpec{
        Replicas: int32Ptr(1),
        Selector: &metav1.LabelSelector{MatchLabels: map[string]string{"app":
"storage-app"}},
        Template: corev1.PodTemplateSpec{
            ObjectMeta: metav1.ObjectMeta{Labels: map[string]string{"app":
"storage-app"}},
            Spec: corev1.PodSpec{
                Containers: []corev1.Container{
                    {
```

```
                        Name:  "app",
                        Image: "myapp:1.0",
                        VolumeMounts: []corev1.VolumeMount{
                            {Name: "data-volume", MountPath: "/var/data"},
                        },
                    },
                },
                Volumes: []corev1.Volume{
                    {
                        Name: "data-volume",
                        VolumeSource: corev1.VolumeSource{
                            PersistentVolumeClaim:
&corev1.PersistentVolumeClaimVolumeSource{
                                ClaimName: "my-data-pvc",
                            },
                        },
                    },
                },
            },
        },
    },
}
```

Delete PVC

```go
// PVC must not be in use by any Pod
err := clientset.CoreV1().PersistentVolumeClaims("default").Delete(ctx, "my-data-pvc", metav1.DeleteOptions{})

// Delete PV manually if reclaimPolicy is Retain
err = clientset.CoreV1().PersistentVolumes().Delete(ctx, "my-nfs-pv", metav1.DeleteOptions{})
```

Storage Capacity Summary

```go
func printStorageSummary(ctx context.Context, clientset *kubernetes.Clientset)
{
```

```
    pvcs, _ := clientset.CoreV1().PersistentVolumeClaims("").List(ctx,
metav1.ListOptions{})

    var totalGi int64
    for _, pvc := range pvcs.Items {
        req := pvc.Spec.Resources.Requests[corev1.ResourceStorage]
        totalGi += req.Value() / (1024 * 1024 * 1024)
        fmt.Printf("%-30s %-15s %-10s %s\n",
            pvc.Name, pvc.Namespace, pvc.Status.Phase, req.String())
    }
    fmt.Printf("\nTotal PVC storage: %dGi across %d PVCs\n", totalGi,
len(pvcs.Items))
}
```

Chapter 9: Namespaces & RBAC

RBAC Interfaces

```go
clientset.CoreV1().Namespaces()
clientset.CoreV1().ServiceAccounts(namespace)
clientset.RbacV1().Roles(namespace)
clientset.RbacV1().ClusterRoles()
clientset.RbacV1().RoleBindings(namespace)
clientset.RbacV1().ClusterRoleBindings()
```

Working with Namespaces

List Namespaces

```go
nsList, err := clientset.CoreV1().Namespaces().List(ctx, metav1.ListOptions{})
for _, ns := range nsList.Items {
    fmt.Printf("[%s] %s\n", ns.Status.Phase, ns.Name)
}
```

Create a Namespace

```go
ns := &corev1.Namespace{
    ObjectMeta: metav1.ObjectMeta{
        Name: "team-alpha",
        Labels: map[string]string{
            "team":        "alpha",
            "environment": "development",
        },
        Annotations: map[string]string{
            "owner": "alice@example.com",
        },
    },
}

created, err := clientset.CoreV1().Namespaces().Create(ctx, ns,
metav1.CreateOptions{})
fmt.Printf("Namespace created: %s\n", created.Name)
```

Delete a Namespace

```go
// Deletes ALL resources inside it
err := clientset.CoreV1().Namespaces().Delete(ctx, "team-alpha",
metav1.DeleteOptions{})
```

Wait for Namespace Termination

```go
func waitForNamespaceGone(ctx context.Context, clientset *kubernetes.Clientset,
name string) error {
    for {
        _, err := clientset.CoreV1().Namespaces().Get(ctx, name,
metav1.GetOptions{})
        if errors.IsNotFound(err) {
            fmt.Printf("Namespace '%s' fully deleted\n", name)
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
            fmt.Printf("Namespace '%s' still terminating...\n", name)
        }
    }
}
```

Working with ServiceAccounts

Create a ServiceAccount

```go
sa := &corev1.ServiceAccount{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "ci-pipeline",
        Namespace: "default",
        Labels:    map[string]string{"purpose": "ci"},
        Annotations: map[string]string{
            "description": "ServiceAccount for GitHub Actions CI pipeline",
        },
    },
}

clientset.CoreV1().ServiceAccounts("default").Create(ctx, sa,
metav1.CreateOptions{})
```

List ServiceAccounts

```go
sas, _ := clientset.CoreV1().ServiceAccounts("default").List(ctx,
metav1.ListOptions{})
for _, sa := range sas.Items {
    fmt.Println(sa.Name)
}
```

Working with Roles

Create a Role

```go
import rbacv1 "k8s.io/api/rbac/v1"

role := &rbacv1.Role{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "pod-reader",
        Namespace: "default",
```

```
    },
    Rules: []rbacv1.PolicyRule{
        {
            APIGroups: []string{""},
            Resources: []string{"pods", "pods/log", "pods/status"},
            Verbs:     []string{"get", "list", "watch"},
        },
    },
}

clientset.RbacV1().Roles("default").Create(ctx, role, metav1.CreateOptions{})
```

Deployment Manager Role

```go
role := &rbacv1.Role{
    ObjectMeta: metav1.ObjectMeta{Name: "deployment-manager", Namespace:
"production"},
    Rules: []rbacv1.PolicyRule{
        {
            APIGroups: []string{"apps"},
            Resources: []string{"deployments", "replicasets"},
            Verbs:     []string{"get", "list", "watch", "create", "update",
"patch"},
        },
        {
            APIGroups: []string{""},
            Resources: []string{"pods", "services", "configmaps"},
            Verbs:     []string{"get", "list", "watch"},
        },
    },
}

clientset.RbacV1().Roles("production").Create(ctx, role,
metav1.CreateOptions{})
```

Working with ClusterRoles

```go
clusterRole := &rbacv1.ClusterRole{
    ObjectMeta: metav1.ObjectMeta{Name: "cluster-reader"},
    Rules: []rbacv1.PolicyRule{
```

```
        {
            APIGroups: []string{"", "apps", "batch"},
            Resources: []string{"*"},
            Verbs:     []string{"get", "list", "watch"},
        },
        {
            APIGroups: []string{""},
            Resources: []string{"nodes", "namespaces", "persistentvolumes"},
            Verbs:     []string{"get", "list"},
        },
    },
}

clientset.RbacV1().ClusterRoles().Create(ctx, clusterRole,
metav1.CreateOptions{})
```

Working with RoleBindings

go

```
roleBinding := &rbacv1.RoleBinding{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "ci-pipeline-binding",
        Namespace: "default",
    },
    Subjects: []rbacv1.Subject{
        {
            Kind:      "ServiceAccount",
            Name:      "ci-pipeline",
            Namespace: "default",
        },
    },
    RoleRef: rbacv1.RoleRef{
        APIGroup: "rbac.authorization.k8s.io",
        Kind:     "Role",
        Name:     "deployment-manager",
    },
}

clientset.RbacV1().RoleBindings("default").Create(ctx, roleBinding,
metav1.CreateOptions{})
```

Bind to a User

```go
roleBinding := &rbacv1.RoleBinding{
    ObjectMeta: metav1.ObjectMeta{Name: "alice-binding", Namespace: "team-alpha"},
    Subjects: []rbacv1.Subject{
        {
            Kind:     "User",
            Name:     "alice@example.com",
            APIGroup: "rbac.authorization.k8s.io",
        },
    },
    RoleRef: rbacv1.RoleRef{
        APIGroup: "rbac.authorization.k8s.io",
        Kind:     "Role",
        Name:     "pod-reader",
    },
}
```

Working with ClusterRoleBindings

```go
crb := &rbacv1.ClusterRoleBinding{
    ObjectMeta: metav1.ObjectMeta{Name: "dashboard-cluster-reader"},
    Subjects: []rbacv1.Subject{
        {
            Kind:      "ServiceAccount",
            Name:      "dashboard",
            Namespace: "kube-system",
        },
    },
    RoleRef: rbacv1.RoleRef{
        APIGroup: "rbac.authorization.k8s.io",
        Kind:     "ClusterRole",
        Name:     "cluster-reader",
    },
}

clientset.RbacV1().ClusterRoleBindings().Create(ctx, crb, metav1.CreateOptions{})
```

Multi-Tenant Namespace Provisioning

```go
func provisionTenantNamespace(ctx context.Context, clientset
*kubernetes.Clientset, team, ownerEmail string) error {
    ns := team + "-workspace"

    // 1. Create namespace
    _, err := clientset.CoreV1().Namespaces().Create(ctx, &corev1.Namespace{
        ObjectMeta: metav1.ObjectMeta{
            Name:        ns,
            Labels:      map[string]string{"team": team, "managed-by":
"provisioner"},
            Annotations: map[string]string{"owner": ownerEmail},
        },
    }, metav1.CreateOptions{})
    if err != nil && !errors.IsAlreadyExists(err) {
        return err
    }

    // 2. ServiceAccount
    clientset.CoreV1().ServiceAccounts(ns).Create(ctx, &corev1.ServiceAccount{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-sa", Namespace: ns},
    }, metav1.CreateOptions{})

    // 3. Role
    clientset.RbacV1().Roles(ns).Create(ctx, &rbacv1.Role{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-role", Namespace: ns},
        Rules: []rbacv1.PolicyRule{{
            APIGroups: []string{"", "apps", "batch"},
            Resources: []string{"*"},
            Verbs:     []string{"*"},
        }},
    }, metav1.CreateOptions{})

    // 4. RoleBinding
    clientset.RbacV1().RoleBindings(ns).Create(ctx, &rbacv1.RoleBinding{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-binding", Namespace: ns},
        Subjects:   []rbacv1.Subject{{Kind: "ServiceAccount", Name: team + "-
sa", Namespace: ns}},
        RoleRef:    rbacv1.RoleRef{APIGroup: "rbac.authorization.k8s.io", Kind:
"Role", Name: team + "-role"},
    }, metav1.CreateOptions{})

    fmt.Printf("Namespace '%s' provisioned for team '%s'\n", ns, team)
```

```
    return nil
}
```

List Roles and Their Rules

```go
roles, _ := clientset.RbacV1().Roles("default").List(ctx, metav1.ListOptions{})
for _, role := range roles.Items {
    fmt.Printf("Role: %s\n", role.Name)
    for _, rule := range role.Rules {
        fmt.Printf("  Resources: %v | Verbs: %v\n", rule.Resources, rule.Verbs)
    }
}
```

Chapter 10: Ingress

Ingress Interface

```go
clientset.NetworkingV1().Ingresses(namespace)
```

Listing Ingresses

```go
import netv1 "k8s.io/api/networking/v1"

ingresses, err := clientset.NetworkingV1().Ingresses("default").List(ctx,
metav1.ListOptions{})
for _, ing := range ingresses.Items {
    for _, rule := range ing.Spec.Rules {
        host := rule.Host
        if rule.HTTP == nil {
            continue
        }
        for _, path := range rule.HTTP.Paths {
            backend := path.Backend.Service
            fmt.Printf("%-20s %-25s %-10s → %s:%d\n",
                ing.Name, host, path.Path,
```

```
                backend.Name, backend.Port.Number)
        }
    }
}
```

Creating a Basic Ingress

```go
import (
    netv1 "k8s.io/api/networking/v1"
)

ingressClass  := "nginx"
pathTypePrefix := netv1.PathTypePrefix

ingress := &netv1.Ingress{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-app-ingress",
        Namespace: "default",
        Annotations: map[string]string{
            "nginx.ingress.kubernetes.io/rewrite-target": "/",
        },
    },
    Spec: netv1.IngressSpec{
        IngressClassName: &ingressClass,
        Rules: []netv1.IngressRule{
            {
                Host: "myapp.example.com",
                IngressRuleValue: netv1.IngressRuleValue{
                    HTTP: &netv1.HTTPIngressRuleValue{
                        Paths: []netv1.HTTPIngressPath{
                            {
                                Path:     "/api",
                                PathType: &pathTypePrefix,
                                Backend: netv1.IngressBackend{
                                    Service: &netv1.IngressServiceBackend{
                                        Name: "api-service",
                                        Port: netv1.ServiceBackendPort{Number:
80},
                                    },
                                },
                            },
```

```
                            {
                                Path:     "/",
                                PathType: &pathTypePrefix,
                                Backend: netv1.IngressBackend{
                                    Service: &netv1.IngressServiceBackend{
                                        Name: "frontend-service",
                                        Port: netv1.ServiceBackendPort{Number:
80},
                                    },
                                },
                            },
                        },
                    },
                },
            },
        },
    },
}

created, err := clientset.NetworkingV1().Ingresses("default").Create(ctx,
ingress, metav1.CreateOptions{})
fmt.Printf("Ingress created: %s\n", created.Name)
```

Creating an Ingress with TLS

go

```
ingress := &netv1.Ingress{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "my-app-tls-ingress",
        Namespace: "default",
        Annotations: map[string]string{
            "cert-manager.io/cluster-issuer":           "letsencrypt-prod",
            "nginx.ingress.kubernetes.io/ssl-redirect": "true",
        },
    },
    Spec: netv1.IngressSpec{
        IngressClassName: &ingressClass,
        TLS: []netv1.IngressTLS{
            {
                Hosts:      []string{"myapp.example.com", "api.example.com"},
                SecretName: "myapp-tls-secret",
            },
```

```
        },
        Rules: []netv1.IngressRule{
            {
                Host: "myapp.example.com",
                IngressRuleValue: netv1.IngressRuleValue{
                    HTTP: &netv1.HTTPIngressRuleValue{
                        Paths: []netv1.HTTPIngressPath{{
                            Path: "/", PathType: &pathTypePrefix,
                            Backend: netv1.IngressBackend{
                                Service: &netv1.IngressServiceBackend{
                                    Name: "frontend", Port:
netv1.ServiceBackendPort{Number: 80},
                                },
                            },
                        }},
                    },
                },
            },
            {
                Host: "api.example.com",
                IngressRuleValue: netv1.IngressRuleValue{
                    HTTP: &netv1.HTTPIngressRuleValue{
                        Paths: []netv1.HTTPIngressPath{{
                            Path: "/", PathType: &pathTypePrefix,
                            Backend: netv1.IngressBackend{
                                Service: &netv1.IngressServiceBackend{
                                    Name: "api", Port:
netv1.ServiceBackendPort{Number: 80},
                                },
                            },
                        }},
                    },
                },
            },
        },
    },
}

clientset.NetworkingV1().Ingresses("default").Create(ctx, ingress,
metav1.CreateOptions{})
```

Getting Ingress Address

```go
ing, err := clientset.NetworkingV1().Ingresses("default").Get(ctx, "my-app-ingress", metav1.GetOptions{})

for _, lb := range ing.Status.LoadBalancer.Ingress {
    addr := lb.IP
    if addr == "" {
        addr = lb.Hostname
    }
    fmt.Printf("Address: %s\n", addr)
}
```

Updating an Ingress (Add a Host)

```go
import "k8s.io/client-go/util/retry"

err = retry.RetryOnConflict(retry.DefaultRetry, func() error {
    ing, err := clientset.NetworkingV1().Ingresses("default").Get(ctx, "my-app-ingress", metav1.GetOptions{})
    if err != nil {
        return err
    }

    ing.Spec.Rules = append(ing.Spec.Rules, netv1.IngressRule{
        Host: "new-service.example.com",
        IngressRuleValue: netv1.IngressRuleValue{
            HTTP: &netv1.HTTPIngressRuleValue{
                Paths: []netv1.HTTPIngressPath{{
                    Path: "/", PathType: &pathTypePrefix,
                    Backend: netv1.IngressBackend{
                        Service: &netv1.IngressServiceBackend{
                            Name: "new-service", Port: netv1.ServiceBackendPort{Number: 80},
                        },
                    },
                }},
            },
        },
    })
```

```
    _, err = clientset.NetworkingV1().Ingresses("default").Update(ctx, ing,
metav1.UpdateOptions{})
    return err
})
```

Delete an Ingress

```go
err := clientset.NetworkingV1().Ingresses("default").Delete(ctx, "my-app-
ingress", metav1.DeleteOptions{})
```

Common Nginx Annotations

```go
annotations := map[string]string{
    // Rewrite
    "nginx.ingress.kubernetes.io/rewrite-target": "/$2",
    // Rate limiting
    "nginx.ingress.kubernetes.io/limit-rps":        "10",
    "nginx.ingress.kubernetes.io/limit-connections": "20",
    // Proxy timeouts
    "nginx.ingress.kubernetes.io/proxy-read-timeout": "300",
    "nginx.ingress.kubernetes.io/proxy-body-size":    "50m",
    // SSL
    "nginx.ingress.kubernetes.io/ssl-redirect":       "true",
    "nginx.ingress.kubernetes.io/force-ssl-redirect": "true",
    // CORS
    "nginx.ingress.kubernetes.io/enable-cors":         "true",
    "nginx.ingress.kubernetes.io/cors-allow-origin":   "https://myapp.com",
}
```

PathType Reference

```go
netv1.PathTypeExact                  // exact path match
netv1.PathTypePrefix                 // prefix match
```

```
netv1.PathTypeImplementationSpecific // controller-specific
```

Chapter 11: StatefulSets & DaemonSets

Interfaces

```go
clientset.AppsV1().StatefulSets(namespace)
clientset.AppsV1().DaemonSets(namespace)
```

Working with StatefulSets

List StatefulSets

```go
stsList, err := clientset.AppsV1().StatefulSets("default").List(ctx,
metav1.ListOptions{})
for _, sts := range stsList.Items {
    fmt.Printf("%-30s  %d/%d ready\n", sts.Name, sts.Status.ReadyReplicas,
*sts.Spec.Replicas)
}
```

Create a StatefulSet (MySQL)

```go
import (
    appsv1 "k8s.io/api/apps/v1"
    corev1 "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/api/resource"
)

storageClass := "standard"

sts := &appsv1.StatefulSet{
    ObjectMeta: metav1.ObjectMeta{Name: "mysql", Namespace: "default"},
    Spec: appsv1.StatefulSetSpec{
        ServiceName: "mysql-headless",
        Replicas:    int32Ptr(3),
        Selector:    &metav1.LabelSelector{MatchLabels:
map[string]string{"app": "mysql"}},
```

```
        Template: corev1.PodTemplateSpec{
            ObjectMeta: metav1.ObjectMeta{Labels: map[string]string{"app":
"mysql"}},
            Spec: corev1.PodSpec{
                Containers: []corev1.Container{
                    {
                        Name:  "mysql",
                        Image: "mysql:8.0",
                        Env: []corev1.EnvVar{
                            {
                                Name: "MYSQL_ROOT_PASSWORD",
                                ValueFrom: &corev1.EnvVarSource{
                                    SecretKeyRef: &corev1.SecretKeySelector{
                                        LocalObjectReference:
corev1.LocalObjectReference{Name: "mysql-secret"},
                                        Key: "root-password",
                                    },
                                },
                            },
                        },
                        Ports: []corev1.ContainerPort{{ContainerPort: 3306,
Name: "mysql"}},
                        VolumeMounts: []corev1.VolumeMount{
                            {Name: "data", MountPath: "/var/lib/mysql"},
                        },
                        Resources: corev1.ResourceRequirements{
                            Requests: corev1.ResourceList{
                                corev1.ResourceCPU:
resource.MustParse("250m"),
                                corev1.ResourceMemory:
resource.MustParse("512Mi"),
                            },
                        },
                        ReadinessProbe: &corev1.Probe{
                            ProbeHandler: corev1.ProbeHandler{
                                Exec: &corev1.ExecAction{
                                    Command: []string{"bash", "-c", "mysqladmin
ping -h 127.0.0.1"},
                                },
                            },
                            InitialDelaySeconds: 20,
                            PeriodSeconds:       10,
                        },
                    },
                },
            },
```

```
        },
        VolumeClaimTemplates: []corev1.PersistentVolumeClaim{
            {
                ObjectMeta: metav1.ObjectMeta{Name: "data"},
                Spec: corev1.PersistentVolumeClaimSpec{
                    AccessModes:
[]corev1.PersistentVolumeAccessMode{corev1.ReadWriteOnce},
                    StorageClassName: &storageClass,
                    Resources: corev1.ResourceRequirements{
                        Requests: corev1.ResourceList{
                            corev1.ResourceStorage: resource.MustParse("20Gi"),
                        },
                    },
                },
            },
        },
    },
}

clientset.AppsV1().StatefulSets("default").Create(ctx, sts,
metav1.CreateOptions{})
// Pods: mysql-0, mysql-1, mysql-2
// DNS:  mysql-0.mysql-headless.default.svc.cluster.local
```

Scale a StatefulSet

go

```
err := retry.RetryOnConflict(retry.DefaultRetry, func() error {
    sts, err := clientset.AppsV1().StatefulSets("default").Get(ctx, "mysql",
metav1.GetOptions{})
    if err != nil {
        return err
    }
    sts.Spec.Replicas = int32Ptr(5)
    _, err = clientset.AppsV1().StatefulSets("default").Update(ctx, sts,
metav1.UpdateOptions{})
    return err
})
```

StatefulSet Pod DNS

go

```
func statefulSetPodDNS(podIndex int, stsName, serviceName, namespace string)
string {
    return fmt.Sprintf("%s-%d.%s.%s.svc.cluster.local", stsName, podIndex,
```

```
serviceName, namespace)
}

sts, _ := clientset.AppsV1().StatefulSets("default").Get(ctx, "mysql",
metav1.GetOptions{})
for i := int32(0); i < *sts.Spec.Replicas; i++ {
    dns := statefulSetPodDNS(int(i), "mysql", sts.Spec.ServiceName, "default")
    fmt.Printf("Replica %d: %s\n", i, dns)
}
```

Update Strategy (Partition / Canary)

```go
partition := int32(2) // only update pods with index >= 2

err := retry.RetryOnConflict(retry.DefaultRetry, func() error {
    sts, err := clientset.AppsV1().StatefulSets("default").Get(ctx, "mysql",
metav1.GetOptions{})
    if err != nil {
        return err
    }
    sts.Spec.UpdateStrategy = appsv1.StatefulSetUpdateStrategy{
        Type: appsv1.RollingUpdateStatefulSetStrategyType,
        RollingUpdate: &appsv1.RollingUpdateStatefulSetStrategy{
            Partition: &partition,
        },
    }
    sts.Spec.Template.Spec.Containers[0].Image = "mysql:8.1"
    _, err = clientset.AppsV1().StatefulSets("default").Update(ctx, sts,
metav1.UpdateOptions{})
    return err
})
```

Working with DaemonSets

List DaemonSets

```go
dsList, err := clientset.AppsV1().DaemonSets("kube-system").List(ctx,
metav1.ListOptions{})
for _, ds := range dsList.Items {
```

```
        fmt.Printf("%-35s desired=%-3d ready=%-3d available=%d\n",
            ds.Name,
            ds.Status.DesiredNumberScheduled,
            ds.Status.NumberReady,
            ds.Status.NumberAvailable,
        )
    }
```

Create a DaemonSet (Fluentd Log Collector)

```go
    ds := &appsv1.DaemonSet{
        ObjectMeta: metav1.ObjectMeta{
            Name:      "fluentd",
            Namespace: "kube-system",
            Labels:    map[string]string{"app": "fluentd"},
        },
        Spec: appsv1.DaemonSetSpec{
            Selector: &metav1.LabelSelector{
                MatchLabels: map[string]string{"app": "fluentd"},
            },
            UpdateStrategy: appsv1.DaemonSetUpdateStrategy{
                Type: appsv1.RollingUpdateDaemonSetStrategyType,
                RollingUpdate: &appsv1.RollingUpdateDaemonSet{
                    MaxUnavailable: &intstr.IntOrString{Type: intstr.Int, IntVal:
    1},
                },
            },
            Template: corev1.PodTemplateSpec{
                ObjectMeta: metav1.ObjectMeta{Labels: map[string]string{"app":
    "fluentd"}},
                Spec: corev1.PodSpec{
                    Tolerations: []corev1.Toleration{
                        {
                            Key:    "node-role.kubernetes.io/control-plane",
                            Effect: corev1.TaintEffectNoSchedule,
                        },
                    },
                    Containers: []corev1.Container{
                        {
                            Name:  "fluentd",
                            Image: "fluent/fluentd-kubernetes-daemonset:v1.16",
                            Resources: corev1.ResourceRequirements{
                                Limits:
    corev1.ResourceList{corev1.ResourceMemory: resource.MustParse("200Mi")},
```

```
                                Requests: corev1.ResourceList{
                                    corev1.ResourceCPU:
resource.MustParse("100m"),
                                    corev1.ResourceMemory:
resource.MustParse("200Mi"),
                                },
                            },
                            VolumeMounts: []corev1.VolumeMount{
                                {Name: "varlog", MountPath: "/var/log"},
                                {Name: "varlibdockercontainers", MountPath:
"/var/lib/docker/containers", ReadOnly: true},
                            },
                        },
                    },
                    Volumes: []corev1.Volume{
                        {Name: "varlog", VolumeSource:
corev1.VolumeSource{HostPath: &corev1.HostPathVolumeSource{Path: "/var/log"}}},
                        {Name: "varlibdockercontainers", VolumeSource:
corev1.VolumeSource{HostPath: &corev1.HostPathVolumeSource{Path:
"/var/lib/docker/containers"}}},
                    },
                },
            },
        },
    },
}

clientset.AppsV1().DaemonSets("kube-system").Create(ctx, ds,
metav1.CreateOptions{})
```

DaemonSet on Specific Nodes Only

```go
ds.Spec.Template.Spec.NodeSelector = map[string]string{
    "gpu": "true",   // only nodes labelled gpu=true
}
```

DaemonSet Health Check

```go
func isDaemonSetHealthy(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string) bool {
    ds, err := clientset.AppsV1().DaemonSets(namespace).Get(ctx, name,
metav1.GetOptions{})
    if err != nil {
        return false
```

```
    }
    return ds.Status.NumberReady == ds.Status.DesiredNumberScheduled &&
ds.Status.DesiredNumberScheduled > 0
}
```

Delete StatefulSet and DaemonSet

go

```
propagation := metav1.DeletePropagationForeground

// Delete StatefulSet (PVCs are NOT deleted automatically)
clientset.AppsV1().StatefulSets("default").Delete(ctx, "mysql",
metav1.DeleteOptions{
    PropagationPolicy: &propagation,
})

// Manually delete PVCs
pvcs, _ := clientset.CoreV1().PersistentVolumeClaims("default").List(ctx,
metav1.ListOptions{
    LabelSelector: "app=mysql",
})
for _, pvc := range pvcs.Items {
    clientset.CoreV1().PersistentVolumeClaims("default").Delete(ctx, pvc.Name,
metav1.DeleteOptions{})
}

// Delete DaemonSet
clientset.AppsV1().DaemonSets("kube-system").Delete(ctx, "fluentd",
metav1.DeleteOptions{})
```

Chapter 12: Jobs & CronJobs

Interfaces

go

```
clientset.BatchV1().Jobs(namespace)
clientset.BatchV1().CronJobs(namespace)
```

Working with Jobs

List Jobs

```go
jobs, err := clientset.BatchV1().Jobs("default").List(ctx,
metav1.ListOptions{})
for _, job := range jobs.Items {
    active    := job.Status.Active
    succeeded := job.Status.Succeeded
    failed    := job.Status.Failed
    fmt.Printf("%-30s active=%-3d succeeded=%-3d failed=%d\n",
        job.Name, active, succeeded, failed)
}
```

Create a Job

```go
import batchv1 "k8s.io/api/batch/v1"

backoffLimit    := int32(3)
ttlSeconds      := int32(3600)
completions     := int32(1)
parallelism     := int32(1)

job := &batchv1.Job{
    ObjectMeta: metav1.ObjectMeta{
        Name:      "db-migration",
        Namespace: "default",
        Labels:    map[string]string{"type": "migration"},
    },
    Spec: batchv1.JobSpec{
        Completions:             &completions,
        Parallelism:             &parallelism,
        BackoffLimit:            &backoffLimit,
        TTLSecondsAfterFinished: &ttlSeconds,
        Template: corev1.PodTemplateSpec{
            ObjectMeta: metav1.ObjectMeta{Labels: map[string]string{"job": "db-
migration"}},
            Spec: corev1.PodSpec{
                RestartPolicy: corev1.RestartPolicyOnFailure,
                Containers: []corev1.Container{
                    {
                        Name:    "migrator",
                        Image:   "myapp:latest",
```

```
                        Command: []string{"go", "run", "migrate.go"},
                        Env: []corev1.EnvVar{
                            {Name: "DB_HOST", Value: "postgres"},
                            {
                                Name: "DB_PASSWORD",
                                ValueFrom: &corev1.EnvVarSource{
                                    SecretKeyRef: &corev1.SecretKeySelector{
                                        LocalObjectReference:
corev1.LocalObjectReference{Name: "db-secret"},
                                        Key: "password",
                                    },
                                },
                            },
                        },
                        Resources: corev1.ResourceRequirements{
                            Requests: corev1.ResourceList{
                                corev1.ResourceCPU:
resource.MustParse("100m"),
                                corev1.ResourceMemory:
resource.MustParse("128Mi"),
                            },
                        },
                    },
                },
            },
        },
    },
}

created, err := clientset.BatchV1().Jobs("default").Create(ctx, job,
metav1.CreateOptions{})
fmt.Printf("Job created: %s\n", created.Name)
```

Wait for Job Completion

go

```
func waitForJob(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string) (string, error) {
    for {
        job, err := clientset.BatchV1().Jobs(namespace).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return "", err
        }
```

```
        if job.Status.Succeeded > 0 {
            return "succeeded", nil
        }

        backoff := *job.Spec.BackoffLimit
        if job.Status.Failed > backoff {
            return "failed", fmt.Errorf("job %s failed after %d attempts",
name, job.Status.Failed)
        }

        fmt.Printf("Job: active=%d succeeded=%d failed=%d\n",
            job.Status.Active, job.Status.Succeeded, job.Status.Failed)

        select {
        case <-ctx.Done():
            return "timeout", ctx.Err()
        case <-time.After(5 * time.Second):
        }
    }
}

timeoutCtx, cancel := context.WithTimeout(context.Background(), 5*time.Minute)
defer cancel()

status, err := waitForJob(timeoutCtx, clientset, "default", "db-migration")
fmt.Printf("Job status: %s\n", status)
```

Parallel Job (Batch Processing)

go

```
completions  := int32(10)
parallelism  := int32(3)
backoffLimit := int32(2)

job := &batchv1.Job{
    ObjectMeta: metav1.ObjectMeta{Name: "batch-processor"},
    Spec: batchv1.JobSpec{
        Completions:  &completions,
        Parallelism:  &parallelism,
        BackoffLimit: &backoffLimit,
        Template: corev1.PodTemplateSpec{
            Spec: corev1.PodSpec{
                RestartPolicy: corev1.RestartPolicyNever,
                Containers: []corev1.Container{{
                    Name:    "worker",
```

```
                        Image:   "myapp:batch",
                        Command: []string{"/app/worker"},
                    }},
                },
            },
        },
    }
```

Delete a Job

```go
propagation := metav1.DeletePropagationForeground

err := clientset.BatchV1().Jobs("default").Delete(ctx, "db-migration",
metav1.DeleteOptions{
    PropagationPolicy: &propagation,
})
```

Working with CronJobs

List CronJobs

```go
cjs, err := clientset.BatchV1().CronJobs("default").List(ctx,
metav1.ListOptions{})
for _, cj := range cjs.Items {
    lastSchedule := "never"
    if cj.Status.LastScheduleTime != nil {
        lastSchedule = cj.Status.LastScheduleTime.String()
    }
    suspended := ""
    if cj.Spec.Suspend != nil && *cj.Spec.Suspend {
        suspended = "SUSPENDED"
    }
    fmt.Printf("%-30s %-15s %-25s active=%-2d %s\n",
        cj.Name, cj.Spec.Schedule, lastSchedule, len(cj.Status.Active),
suspended)
}
```

Create a CronJob

go

```
successLimit := int32(3)
failLimit    := int32(1)
suspend      := false
startDeadline := int64(300)

cronJob := &batchv1.CronJob{
    ObjectMeta: metav1.ObjectMeta{Name: "daily-report", Namespace: "default"},
    Spec: batchv1.CronJobSpec{
        Schedule:                   "0 6 * * *",
        ConcurrencyPolicy:          batchv1.ForbidConcurrent,
        SuccessfulJobsHistoryLimit: &successLimit,
        FailedJobsHistoryLimit:     &failLimit,
        Suspend:                    &suspend,
        StartingDeadlineSeconds:    &startDeadline,
        JobTemplate: batchv1.JobTemplateSpec{
            Spec: batchv1.JobSpec{
                BackoffLimit: int32Ptr(2),
                Template: corev1.PodTemplateSpec{
                    Spec: corev1.PodSpec{
                        RestartPolicy: corev1.RestartPolicyOnFailure,
                        Containers: []corev1.Container{{
                            Name:    "reporter",
                            Image:   "myapp:latest",
                            Command: []string{"go", "run", "report.go"},
                            Resources: corev1.ResourceRequirements{
                                Requests: corev1.ResourceList{
                                    corev1.ResourceCPU:
resource.MustParse("200m"),
                                    corev1.ResourceMemory:
resource.MustParse("256Mi"),
                                },
                            },
                        }},
                    },
                },
            },
        },
    },
}

clientset.BatchV1().CronJobs("default").Create(ctx, cronJob,
metav1.CreateOptions{})
```

Suspend / Resume a CronJob

```go
import (
    "encoding/json"
    "k8s.io/apimachinery/pkg/types"
)

type suspendPatch struct {
    Spec struct {
        Suspend bool `json:"suspend"`
    } `json:"spec"`
}

func setCronJobSuspend(ctx context.Context, clientset *kubernetes.Clientset,
namespace, name string, suspend bool) error {
    p := suspendPatch{}
    p.Spec.Suspend = suspend
    data, err := json.Marshal(p)
    if err != nil {
        return err
    }
    _, err = clientset.BatchV1().CronJobs(namespace).Patch(
        ctx, name, types.MergePatchType, data, metav1.PatchOptions{})
    return err
}

setCronJobSuspend(ctx, clientset, "default", "daily-report", true)  // pause
setCronJobSuspend(ctx, clientset, "default", "daily-report", false) // resume
```

Manually Trigger a CronJob

```go
func triggerCronJob(ctx context.Context, clientset *kubernetes.Clientset,
namespace, cronJobName string) (string, error) {
    cj, err := clientset.BatchV1().CronJobs(namespace).Get(ctx, cronJobName,
metav1.GetOptions{})
    if err != nil {
        return "", err
    }

    jobName := fmt.Sprintf("%s-manual-%d", cronJobName, time.Now().Unix())
    job := &batchv1.Job{
        ObjectMeta: metav1.ObjectMeta{
            Name:        jobName,
            Namespace:   namespace,
            Annotations: map[string]string{"triggered-by": "manual"},
        },
```

```
        Spec: cj.Spec.JobTemplate.Spec,
    }

    created, err := clientset.BatchV1().Jobs(namespace).Create(ctx, job,
metav1.CreateOptions{})
    if err != nil {
        return "", err
    }

    fmt.Printf("Triggered job: %s\n", created.Name)
    return created.Name, nil
}
```

ConcurrencyPolicy Reference

```go
batchv1.AllowConcurrent    // multiple instances allowed (default)
batchv1.ForbidConcurrent   // skip if previous still running
batchv1.ReplaceConcurrent  // cancel previous, start new
```

Chapter 13: Informers & Watching

Watch vs Informer

	Raw Watch	SharedInformer
Local cache	No	Yes (synced)
Handles reconnection	Manual	Automatic
Multiple handlers	No	Yes
Event deduplication	No	Yes
Use case	Simple one-off watch	Controllers, operators

Raw Watch

```go
watcher, err := clientset.CoreV1().Pods("default").Watch(ctx,
metav1.ListOptions{
    LabelSelector: "app=my-app",
})
if err != nil {
    log.Fatal(err)
}
defer watcher.Stop()

for event := range watcher.ResultChan() {
    pod, ok := event.Object.(*corev1.Pod)
    if !ok {
        continue
    }
    fmt.Printf("[%s] Pod: %s Phase: %s\n", event.Type, pod.Name,
pod.Status.Phase)
    // event.Type: watch.Added | watch.Modified | watch.Deleted | watch.Error
}
```

SharedInformerFactory (Recommended)

The `SharedInformerFactory` manages informers for all resource types and shares a single watch stream.

```go
import (
    "k8s.io/client-go/informers"
    "k8s.io/client-go/tools/cache"
)

// Create factory with 30-second resync period
factory := informers.NewSharedInformerFactory(clientset, 30*time.Second)

// Get a pod informer
podInformer := factory.Core().V1().Pods().Informer()

// Register event handlers
podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        pod := obj.(*corev1.Pod)
```

```
            fmt.Printf("[ADDED] Pod: %s\n", pod.Name)
        },
        UpdateFunc: func(oldObj, newObj interface{}) {
            pod := newObj.(*corev1.Pod)
            fmt.Printf("[UPDATED] Pod: %s Phase: %s\n", pod.Name, pod.Status.Phase)
        },
        DeleteFunc: func(obj interface{}) {
            pod, ok := obj.(*corev1.Pod)
            if !ok {
                // Handle tombstone
                tombstone, ok := obj.(cache.DeletedFinalStateUnknown)
                if !ok {
                    return
                }
                pod = tombstone.Obj.(*corev1.Pod)
            }
            fmt.Printf("[DELETED] Pod: %s\n", pod.Name)
        },
    })

    // Start the factory (begins syncing)
    stopCh := make(chan struct{})
    defer close(stopCh)

    factory.Start(stopCh)

    // Wait until caches are synced (initial list complete)
    if !cache.WaitForCacheSync(stopCh, podInformer.HasSynced) {
        log.Fatal("Timed out waiting for caches to sync")
    }

    fmt.Println("Cache synced — watching for events...")
    <-stopCh
```

Namespace-Scoped Informer

```
go
// Only watch a specific namespace
factory := informers.NewSharedInformerFactoryWithOptions(
    clientset,
    30*time.Second,
```

```
    informers.WithNamespace("production"),
)
```

Using a Lister (Read from Cache)

```go
// After the informer syncs, use Lister for fast local reads (no API calls)
podLister := factory.Core().V1().Pods().Lister()

// List all pods from cache
pods, err := podLister.List(labels.Everything())

// List pods in a namespace
pods, err := podLister.Pods("default").List(labels.Everything())

// Get a specific pod
pod, err := podLister.Pods("default").Get("my-pod")
```

Informers for Different Resources

```go
factory := informers.NewSharedInformerFactory(clientset, 30*time.Second)

// All common informers
podInformer        := factory.Core().V1().Pods().Informer()
deployInformer     := factory.Apps().V1().Deployments().Informer()
svcInformer        := factory.Core().V1().Services().Informer()
cmInformer         := factory.Core().V1().ConfigMaps().Informer()
secretInformer     := factory.Core().V1().Secrets().Informer()
nsInformer         := factory.Core().V1().Namespaces().Informer()
nodeInformer       := factory.Core().V1().Nodes().Informer()
stsInformer        := factory.Apps().V1().StatefulSets().Informer()
dsInformer         := factory.Apps().V1().DaemonSets().Informer()
jobInformer        := factory.Batch().V1().Jobs().Informer()
cronJobInformer    := factory.Batch().V1().CronJobs().Informer()
ingressInformer    := factory.Networking().V1().Ingresses().Informer()
```

WorkQueue — Controller Pattern

The standard pattern for production controllers: informer enqueues → worker dequeues → reconcile.

go

```
import "k8s.io/client-go/util/workqueue"

type Controller struct {
    clientset   *kubernetes.Clientset
    podInformer cache.SharedIndexInformer
    podLister   listersv1.PodNamespaceLister
    queue       workqueue.RateLimitingInterface
}

func NewController(clientset *kubernetes.Clientset, factory
informers.SharedInformerFactory) *Controller {
    podInformer := factory.Core().V1().Pods()
    queue       :=
workqueue.NewRateLimitingQueue(workqueue.DefaultControllerRateLimiter())

    c := &Controller{
        clientset:   clientset,
        podInformer: podInformer.Informer(),
        podLister:   podInformer.Lister().Pods("default"),
        queue:       queue,
    }

    c.podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
        AddFunc:    func(obj interface{}) { c.enqueue(obj) },
        UpdateFunc: func(_, obj interface{}) { c.enqueue(obj) },
        DeleteFunc: func(obj interface{}) { c.enqueue(obj) },
    })

    return c
}

func (c *Controller) enqueue(obj interface{}) {
    key, err := cache.MetaNamespaceKeyFunc(obj)
    if err != nil {
        return
    }
    c.queue.Add(key)
}

func (c *Controller) Run(workers int, stopCh <-chan struct{}) {
```

```
    defer c.queue.ShutDown()

    if !cache.WaitForCacheSync(stopCh, c.podInformer.HasSynced) {
        log.Fatal("Cache sync failed")
    }

    for i := 0; i < workers; i++ {
        go c.runWorker()
    }

    <-stopCh
}

func (c *Controller) runWorker() {
    for c.processNextItem() {}
}

func (c *Controller) processNextItem() bool {
    key, quit := c.queue.Get()
    if quit {
        return false
    }
    defer c.queue.Done(key)

    err := c.reconcile(key.(string))
    if err != nil {
        // Retry up to 5 times
        if c.queue.NumRequeues(key) < 5 {
            c.queue.AddRateLimited(key)
            return true
        }
        c.queue.Forget(key)
        log.Printf("Dropping key %s after max retries: %v", key, err)
        return true
    }

    c.queue.Forget(key)
    return true
}

func (c *Controller) reconcile(key string) error {
    namespace, name, err := cache.SplitMetaNamespaceKey(key)
    if err != nil {
        return err
    }
```

```
    pod, err := c.podLister.Get(name)
    if errors.IsNotFound(err) {
        fmt.Printf("Pod %s/%s was deleted\n", namespace, name)
        return nil
    }
    if err != nil {
        return err
    }

    fmt.Printf("Reconciling pod %s/%s phase=%s\n", namespace, name,
pod.Status.Phase)
    // Your reconciliation logic here
    return nil
}
```

Watching Events (kubectl get events equivalent)

```go
eventInformer := factory.Core().V1().Events().Informer()

eventInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        event := obj.(*corev1.Event)
        if event.Type == corev1.EventTypeWarning {
            fmt.Printf("[WARNING] %s/%s — %s: %s\n",
                event.InvolvedObject.Kind,
                event.InvolvedObject.Name,
                event.Reason,
                event.Message,
            )
        }
    },
})
```

Label Selector in Watch

```go
import "k8s.io/apimachinery/pkg/labels"
```

```
selector := labels.Set{"app": "nginx"}.String()

watcher, _ := clientset.CoreV1().Pods("default").Watch(ctx, metav1.ListOptions{
    LabelSelector: selector,
})
```

Detecting Restart Storms

```go
podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    UpdateFunc: func(oldObj, newObj interface{}) {
        pod    := newObj.(*corev1.Pod)
        oldPod := oldObj.(*corev1.Pod)

        for i, cs := range pod.Status.ContainerStatuses {
            if i < len(oldPod.Status.ContainerStatuses) {
                oldCS := oldPod.Status.ContainerStatuses[i]
                if cs.RestartCount > oldCS.RestartCount {
                    fmt.Printf("[RESTART] Pod=%s Container=%s restarts=%d\n",
                        pod.Name, cs.Name, cs.RestartCount)
                }
            }
            if cs.State.Waiting != nil && cs.State.Waiting.Reason ==
"CrashLoopBackOff" {
                fmt.Printf("[CRASHLOOP] Pod=%s Container=%s\n", pod.Name,
cs.Name)
            }
        }
    },
})
```

Chapter 14: Custom Resources (CRDs)

Three Ways to Access CRDs

Approach	Use When	Type Safety
Dynamic client	Unknown schema, generic tools	None (unstructured)
Generated typed client	Known CRD, production operator	Full

Approach	Use When	Type Safety
controller-runtime	Building operators	Full + helpers

Dynamic Client

The dynamic client works with any resource — standard or custom — as `unstructured.Unstructured`.

```go
import (
    "k8s.io/client-go/dynamic"
    "k8s.io/apimachinery/pkg/apis/meta/v1/unstructured"
    "k8s.io/apimachinery/pkg/runtime/schema"
)

dynClient, err := dynamic.NewForConfig(config)
if err != nil {
    log.Fatal(err)
}
```

List Custom Resources

```go
// Define the GVR (GroupVersionResource)
certificateGVR := schema.GroupVersionResource{
    Group:    "cert-manager.io",
    Version:  "v1",
    Resource: "certificates",
}

list, err := dynClient.Resource(certificateGVR).Namespace("default").List(ctx,
metav1.ListOptions{})
if err != nil {
    log.Fatal(err)
}

for _, item := range list.Items {
    name := item.GetName()
    // Access spec fields
    dnsNames, _, _ := unstructured.NestedStringSlice(item.Object, "spec",
"dnsNames")
```

```
        ready := "Unknown"
        conditions, _, _ := unstructured.NestedSlice(item.Object, "status",
    "conditions")
        for _, c := range conditions {
            cond := c.(map[string]interface{})
            if cond["type"] == "Ready" {
                ready = cond["status"].(string)
            }
        }
        fmt.Printf("%-30s ready=%-8s dnsNames=%v\n", name, ready, dnsNames)
    }
```

Get a Custom Resource

```go
cert, err := dynClient.Resource(certificateGVR).Namespace("default").Get(ctx,
"myapp-tls", metav1.GetOptions{})

fmt.Println("Name:", cert.GetName())
fmt.Println("Labels:", cert.GetLabels())

secretName, _, _ := unstructured.NestedString(cert.Object, "spec",
"secretName")
fmt.Println("SecretName:", secretName)
```

Create a Custom Resource

```go
cert := &unstructured.Unstructured{
    Object: map[string]interface{}{
        "apiVersion": "cert-manager.io/v1",
        "kind":       "Certificate",
        "metadata": map[string]interface{}{
            "name":      "myapp-tls",
            "namespace": "default",
        },
        "spec": map[string]interface{}{
            "secretName": "myapp-tls-secret",
            "issuerRef": map[string]interface{}{
                "name": "letsencrypt-prod",
                "kind": "ClusterIssuer",
            },
            "dnsNames": []interface{}{
                "myapp.example.com",
                "www.myapp.example.com",
```

```
            },
            "duration":    "2160h",
            "renewBefore": "360h",
        },
    },
}

created, err :=
dynClient.Resource(certificateGVR).Namespace("default").Create(ctx, cert,
metav1.CreateOptions{})
fmt.Printf("Certificate created: %s\n", created.GetName())
```

Update a Custom Resource

```go
import "k8s.io/client-go/util/retry"

err = retry.RetryOnConflict(retry.DefaultRetry, func() error {
    cert, err :=
dynClient.Resource(certificateGVR).Namespace("default").Get(ctx, "myapp-tls",
metav1.GetOptions{})
    if err != nil {
        return err
    }

    // Update a field
    unstructured.SetNestedField(cert.Object, "720h", "spec", "renewBefore")

    _, err =
dynClient.Resource(certificateGVR).Namespace("default").Update(ctx, cert,
metav1.UpdateOptions{})
    return err
})
```

Delete a Custom Resource

```go
err := dynClient.Resource(certificateGVR).Namespace("default").Delete(ctx,
"myapp-tls", metav1.DeleteOptions{})
```

Listing CRDs

```go
import (
    apiextv1 "k8s.io/apiextensions-apiserver/pkg/apis/apiextensions/v1"
    apiextclient "k8s.io/apiextensions-
apiserver/pkg/client/clientset/clientset"
)

// go get k8s.io/apiextensions-apiserver@v0.29.0
apiextCS, err := apiextclient.NewForConfig(config)

crds, err := apiextCS.ApiextensionsV1().CustomResourceDefinitions().List(ctx,
metav1.ListOptions{})
for _, crd := range crds.Items {
    fmt.Printf("%-55s %-30s %s\n", crd.Name, crd.Spec.Group, crd.Spec.Scope)
}
```

Defining a CRD via Go

```go
import apiextv1 "k8s.io/apiextensions-apiserver/pkg/apis/apiextensions/v1"

crd := &apiextv1.CustomResourceDefinition{
    ObjectMeta: metav1.ObjectMeta{Name: "myapps.mycompany.io"},
    Spec: apiextv1.CustomResourceDefinitionSpec{
        Group: "mycompany.io",
        Scope: apiextv1.NamespaceScoped,
        Names: apiextv1.CustomResourceDefinitionNames{
            Plural:   "myapps",
            Singular: "myapp",
            Kind:     "MyApp",
        },
        Versions: []apiextv1.CustomResourceDefinitionVersion{{
            Name:    "v1",
            Served:  true,
            Storage: true,
            Schema: &apiextv1.CustomResourceValidation{
                OpenAPIV3Schema: &apiextv1.JSONSchemaProps{
                    Type: "object",
                    Properties: map[string]apiextv1.JSONSchemaProps{
                        "spec": {
                            Type: "object",
```

```
                                Properties: map[string]apiextv1.JSONSchemaProps{
                                    "replicas": {Type: "integer"},
                                    "image":    {Type: "string"},
                                },
                            },
                        },
                    },
                },
            }},
        },
    }

    apiextCS.ApiextensionsV1().CustomResourceDefinitions().Create(ctx, crd,
    metav1.CreateOptions{})
```

Watch Custom Resources

go

```
watcher, err :=
dynClient.Resource(certificateGVR).Namespace("default").Watch(ctx,
metav1.ListOptions{})
if err != nil {
    log.Fatal(err)
}
defer watcher.Stop()

for event := range watcher.ResultChan() {
    obj  := event.Object.(*unstructured.Unstructured)
    name := obj.GetName()

    ready := "Unknown"
    conditions, _, _ := unstructured.NestedSlice(obj.Object, "status",
"conditions")
    for _, c := range conditions {
        if cmap, ok := c.(map[string]interface{}); ok {
            if cmap["type"] == "Ready" {
                ready = fmt.Sprintf("%v", cmap["status"])
            }
        }
    }
```

```
    fmt.Printf("[%s] Certificate: %s — Ready: %s\n", event.Type, name, ready)
}
```

ArgoCD Application via Dynamic Client

```go
argoAppGVR := schema.GroupVersionResource{
    Group:    "argoproj.io",
    Version:  "v1alpha1",
    Resource: "applications",
}

app := &unstructured.Unstructured{
    Object: map[string]interface{}{
        "apiVersion": "argoproj.io/v1alpha1",
        "kind":       "Application",
        "metadata": map[string]interface{}{
            "name":      "my-app",
            "namespace": "argocd",
        },
        "spec": map[string]interface{}{
            "project": "default",
            "source": map[string]interface{}{
                "repoURL":        "https://github.com/myorg/myrepo",
                "targetRevision": "HEAD",
                "path":           "k8s/production",
            },
            "destination": map[string]interface{}{
                "server":    "https://kubernetes.default.svc",
                "namespace": "production",
            },
            "syncPolicy": map[string]interface{}{
                "automated": map[string]interface{}{
                    "prune":    true,
                    "selfHeal": true,
                },
            },
        },
    },
}
```

```
dynClient.Resource(argoAppGVR).Namespace("argocd").Create(ctx, app,
metav1.CreateOptions{})
```

Common GVRs

```go
// cert-manager
schema.GroupVersionResource{Group: "cert-manager.io", Version: "v1", Resource:
"certificates"}
schema.GroupVersionResource{Group: "cert-manager.io", Version: "v1", Resource:
"clusterissuers"}

// ArgoCD
schema.GroupVersionResource{Group: "argoproj.io", Version: "v1alpha1",
Resource: "applications"}

// Prometheus Operator
schema.GroupVersionResource{Group: "monitoring.coreos.com", Version: "v1",
Resource: "prometheusrules"}

// Flux
schema.GroupVersionResource{Group: "helm.toolkit.fluxcd.io", Version:
"v2beta1", Resource: "helmreleases"}

// Traefik
schema.GroupVersionResource{Group: "traefik.containo.us", Version: "v1alpha1",
Resource: "ingressroutes"}

// VPA
schema.GroupVersionResource{Group: "autoscaling.k8s.io", Version: "v1",
Resource: "verticalpodautoscalers"}
```

Chapter 15: Error Handling & Retry Logic

Kubernetes Error Package

```go
import "k8s.io/apimachinery/pkg/api/errors"
```

Function	Checks For
`errors.IsNotFound(err)`	404 — resource doesn't exist
`errors.IsAlreadyExists(err)`	409 — resource already exists
`errors.IsConflict(err)`	409 — optimistic lock (stale resourceVersion)
`errors.IsForbidden(err)`	403 — no RBAC permission
`errors.IsUnauthorized(err)`	401 — bad/expired credentials
`errors.IsInvalid(err)`	422 — spec validation failed
`errors.IsTooManyRequests(err)`	429 — rate limited
`errors.IsServerTimeout(err)`	503/504 — server timeout
`errors.IsTimeout(err)`	Client-side timeout
`errors.IsServiceUnavailable(err)`	503 — server unavailable

Basic Error Handling

go

```go
pod, err := clientset.CoreV1().Pods("default").Get(ctx, "my-pod",
metav1.GetOptions{})
if err != nil {
    switch {
    case errors.IsNotFound(err):
        fmt.Println("Pod not found")
    case errors.IsForbidden(err):
        fmt.Println("Permission denied — check RBAC")
    case errors.IsUnauthorized(err):
        fmt.Println("Invalid credentials — refresh token")
    default:
        log.Fatalf("Unexpected error: %v", err)
    }
    return
}
```

RetryOnConflict

The most important retry pattern — handles 409 Conflict from stale resourceVersion:

```go
import "k8s.io/client-go/util/retry"

err := retry.RetryOnConflict(retry.DefaultRetry, func() error {
    // Always GET first to get the latest resourceVersion
    dep, err := clientset.AppsV1().Deployments("default").Get(ctx, "my-app",
metav1.GetOptions{})
    if err != nil {
        return err
    }

    // Modify
    dep.Spec.Replicas = int32Ptr(5)

    // Update — may fail with 409 if another writer updated concurrently
    _, err = clientset.AppsV1().Deployments("default").Update(ctx, dep,
metav1.UpdateOptions{})
    return err
})
```

DefaultRetry Settings

```go
retry.DefaultRetry = wait.Backoff{
    Duration: 10 * time.Millisecond,
    Factor:   1.0,
    Jitter:   0.1,
    Steps:    5,
}

// More aggressive retry
retry.RetryOnConflict(wait.Backoff{
    Duration: 100 * time.Millisecond,
    Factor:   2.0,
    Jitter:   0.1,
    Steps:    10,
}, func() error {
    // ...
    return nil
})
```

Custom Retry with Backoff

```go
import (
    "k8s.io/apimachinery/pkg/util/wait"
)

func withRetry(ctx context.Context, maxAttempts int, fn func() error) error {
    backoff := wait.Backoff{
        Duration: 500 * time.Millisecond,
        Factor:   2.0,
        Jitter:   0.1,
        Steps:    maxAttempts,
    }

    return wait.ExponentialBackoffWithContext(ctx, backoff, func(ctx
context.Context) (bool, error) {
        err := fn()
        if err == nil {
            return true, nil
        }

        // Retry on transient errors
        if errors.IsServerTimeout(err) || errors.IsServiceUnavailable(err) ||
errors.IsTooManyRequests(err) {
            return false, nil  // retry
        }

        // Don't retry on permanent errors
        return false, err
    })
}

// Usage
err := withRetry(ctx, 5, func() error {
    _, err := clientset.CoreV1().Pods("default").List(ctx,
metav1.ListOptions{})
    return err
})
```

Apply Pattern (Create or Update)

```go
func applyDeployment(ctx context.Context, clientset *kubernetes.Clientset, dep *appsv1.Deployment) error {
    _, err := clientset.AppsV1().Deployments(dep.Namespace).Get(ctx, dep.Name, metav1.GetOptions{})

    if errors.IsNotFound(err) {
        _, err = clientset.AppsV1().Deployments(dep.Namespace).Create(ctx, dep, metav1.CreateOptions{})
        return err
    }
    if err != nil {
        return err
    }

    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        existing, err := clientset.AppsV1().Deployments(dep.Namespace).Get(ctx, dep.Name, metav1.GetOptions{})
        if err != nil {
            return err
        }
        dep.ResourceVersion = existing.ResourceVersion
        _, err = clientset.AppsV1().Deployments(dep.Namespace).Update(ctx, dep, metav1.UpdateOptions{})
        return err
    })
}
```

Delete If Exists

```go
func deleteIfExists(ctx context.Context, clientset *kubernetes.Clientset, namespace, name string) error {
    err := clientset.AppsV1().Deployments(namespace).Delete(ctx, name, metav1.DeleteOptions{})
    if errors.IsNotFound(err) {
        return nil  // already gone — that's fine
    }
    return err
}
```

Context Timeout

```go
// Always use context timeouts for API calls
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

pods, err := clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{})
if err != nil {
    if ctx.Err() != nil {
        fmt.Println("API call timed out")
    } else {
        fmt.Printf("API error: %v\n", err)
    }
}
```

Structured Error Logging

```go
import (
    "k8s.io/apimachinery/pkg/api/errors"
    "k8s.io/apimachinery/pkg/runtime/schema"
)

func logK8sError(operation, resource, name string, err error) {
    if statusErr, ok := err.(*errors.StatusError); ok {
        fmt.Printf("K8s error: op=%s resource=%s name=%s status=%d reason=%s
message=%s\n",
            operation, resource, name,
            statusErr.Status().Code,
            statusErr.Status().Reason,
            statusErr.Status().Message,
        )
    } else {
        fmt.Printf("Error: op=%s resource=%s name=%s err=%v\n", operation,
resource, name, err)
    }
}
```

Handling NotFound in Controllers

```go
func (c *Controller) reconcile(key string) error {
    namespace, name, err := cache.SplitMetaNamespaceKey(key)
    if err != nil {
        return err
    }

    pod, err := c.podLister.Pods(namespace).Get(name)
    if errors.IsNotFound(err) {
        // Pod was deleted — clean up any dependent resources
        fmt.Printf("Pod %s/%s deleted, cleaning up\n", namespace, name)
        return nil  // not an error — expected in deletion flow
    }
    if err != nil {
        return err  // will be retried
    }

    // Reconcile the existing pod
    return c.syncPod(pod)
}
```

Server-Side Apply (SSA)

```go
import (
    "encoding/json"
    "k8s.io/apimachinery/pkg/types"
)

// Server-Side Apply — the safest way to manage resources
// (no read-modify-write, no conflict on resourceVersion)
patchData := map[string]interface{}{
    "apiVersion": "v1",
    "kind":       "ConfigMap",
    "metadata":   map[string]interface{}{"name": "app-config", "namespace":
"default"},
    "data":       map[string]interface{}{"KEY": "value"},
}

data, _ := json.Marshal(patchData)
```

```
_, err = clientset.CoreV1().ConfigMaps("default").Patch(
    ctx,
    "app-config",
    types.ApplyPatchType,
    data,
    metav1.PatchOptions{
        FieldManager: "my-controller",
        Force:        boolPtr(true),
    },
)
```

Common Error Patterns

```go
// Pattern: Create only if not exists
_, err := clientset.CoreV1().Namespaces().Create(ctx, ns,
metav1.CreateOptions{})
if err != nil && !errors.IsAlreadyExists(err) {
    return err
}

// Pattern: Update with conflict retry
err = retry.RetryOnConflict(retry.DefaultRetry, func() error { ... })

// Pattern: Delete idempotently
err = clientset.AppsV1().Deployments(ns).Delete(ctx, name,
metav1.DeleteOptions{})
if err != nil && !errors.IsNotFound(err) {
    return err
}

// Pattern: Check RBAC before proceeding
_, err = clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{})
if errors.IsForbidden(err) {
    log.Fatal("Need permission: pods/list in default namespace")
}
```

Chapter 16: Production Patterns

1. Controller Pattern (Full Example)

The standard Go controller loop: watch → enqueue → reconcile.

go

```
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    corev1  "k8s.io/api/core/v1"
    "k8s.io/apimachinery/pkg/api/errors"
    "k8s.io/apimachinery/pkg/util/wait"
    "k8s.io/client-go/informers"
    "k8s.io/client-go/kubernetes"
    listersv1 "k8s.io/client-go/listers/core/v1"
    "k8s.io/client-go/tools/cache"
    "k8s.io/client-go/util/workqueue"
)

type PodController struct {
    clientset *kubernetes.Clientset
    lister    listersv1.PodLister
    synced    cache.InformerSynced
    queue     workqueue.RateLimitingInterface
}

func NewPodController(clientset *kubernetes.Clientset, factory
informers.SharedInformerFactory) *PodController {
    podInformer := factory.Core().V1().Pods()
    c := &PodController{
        clientset: clientset,
        lister:    podInformer.Lister(),
        synced:    podInformer.Informer().HasSynced,
        queue:
workqueue.NewNamedRateLimitingQueue(workqueue.DefaultControllerRateLimiter(),
"pods"),
    }

    podInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
        AddFunc:    func(obj interface{}) { c.enqueue(obj) },
        UpdateFunc: func(_, obj interface{}) { c.enqueue(obj) },
        DeleteFunc: func(obj interface{}) { c.enqueue(obj) },
    })

    return c
```

```
}

func (c *PodController) enqueue(obj interface{}) {
    key, err := cache.MetaNamespaceKeyFunc(obj)
    if err != nil {
        return
    }
    c.queue.Add(key)
}

func (c *PodController) Run(ctx context.Context, workers int) {
    defer c.queue.ShutDown()
    log.Println("Starting PodController")

    if !cache.WaitForCacheSync(ctx.Done(), c.synced) {
        log.Fatal("Timed out waiting for caches to sync")
    }
    log.Println("Cache synced")

    for i := 0; i < workers; i++ {
        go wait.UntilWithContext(ctx, c.runWorker, time.Second)
    }

    <-ctx.Done()
    log.Println("Shutting down PodController")
}

func (c *PodController) runWorker(ctx context.Context) {
    for c.processNext(ctx) {}
}

func (c *PodController) processNext(ctx context.Context) bool {
    key, quit := c.queue.Get()
    if quit {
        return false
    }
    defer c.queue.Done(key)

    if err := c.reconcile(ctx, key.(string)); err != nil {
        if c.queue.NumRequeues(key) < 5 {
            log.Printf("Retrying %s: %v", key, err)
            c.queue.AddRateLimited(key)
            return true
        }
        log.Printf("Dropping %s after max retries: %v", key, err)
    }
```

```
    c.queue.Forget(key)
    return true
}

func (c *PodController) reconcile(ctx context.Context, key string) error {
    namespace, name, _ := cache.SplitMetaNamespaceKey(key)

    pod, err := c.lister.Pods(namespace).Get(name)
    if errors.IsNotFound(err) {
        log.Printf("Pod %s/%s deleted", namespace, name)
        return nil
    }
    if err != nil {
        return err
    }

    log.Printf("Reconciling pod %s/%s phase=%s", namespace, name,
pod.Status.Phase)
    // Your business logic here
    return nil
}

func main() {
    config, _ := rest.InClusterConfig()
    clientset, _ := kubernetes.NewForConfig(config)

    factory := informers.NewSharedInformerFactory(clientset, 30*time.Second)
    ctrl := NewPodController(clientset, factory)

    ctx, cancel := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    factory.Start(ctx.Done())
    ctrl.Run(ctx, 2) // 2 worker goroutines
}
```

---

2. Leader Election

Only one instance of your controller runs at a time across multiple replicas:

go

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

func runWithLeaderElection(ctx context.Context, clientset
*kubernetes.Clientset, id string, run func(ctx context.Context)) {
    lock := &resourcelock.LeaseLock{
        LeaseMeta: metav1.ObjectMeta{
            Name:      "my-controller-lock",
            Namespace: "default",
        },
        Client: clientset.CoordinationV1(),
        LockConfig: resourcelock.ResourceLockConfig{
            Identity: id,
        },
    }

    leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
        Lock:            lock,
        ReleaseOnCancel: true,
        LeaseDuration:   15 * time.Second,
        RenewDeadline:   10 * time.Second,
        RetryPeriod:     2 * time.Second,
        Callbacks: leaderelection.LeaderCallbacks{
            OnStartedLeading: func(ctx context.Context) {
                fmt.Printf("I am now the leader: %s\n", id)
                run(ctx)
            },
            OnStoppedLeading: func() {
                fmt.Printf("Lost leadership: %s\n", id)
            },
            OnNewLeader: func(identity string) {
                if identity == id {
                    return
                }
                fmt.Printf("New leader elected: %s\n", identity)
            },
        },
    })
}

// Usage — each pod gets a unique ID
podName := os.Getenv("POD_NAME")
runWithLeaderElection(ctx, clientset, podName, func(ctx context.Context) {
```

```
    ctrl.Run(ctx, 2)
})
```

---

3. Deployment Pipeline

```go
type Deployer struct {
    clientset *kubernetes.Clientset
}

func (d *Deployer) Deploy(ctx context.Context, namespace, deployment,
container, image string) error {
    fmt.Printf("Deploying %s to %s/%s...\n", image, namespace, deployment)

    if err := d.updateImage(ctx, namespace, deployment, container, image); err
!= nil {
        return fmt.Errorf("image update failed: %w", err)
    }

    rolloutCtx, cancel := context.WithTimeout(ctx, 5*time.Minute)
    defer cancel()

    if err := d.waitForRollout(rolloutCtx, namespace, deployment); err != nil {
        fmt.Println("Rollout failed — rolling back")
        d.rollback(ctx, namespace, deployment)
        return fmt.Errorf("rollout failed: %w", err)
    }

    fmt.Printf("Deploy of %s succeeded\n", image)
    return nil
}

func (d *Deployer) updateImage(ctx context.Context, ns, name, container, image
string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := d.clientset.AppsV1().Deployments(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        for i, c := range dep.Spec.Template.Spec.Containers {
            if c.Name == container {
                dep.Spec.Template.Spec.Containers[i].Image = image
```

```
                break
            }
        }
        _, err = d.clientset.AppsV1().Deployments(ns).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
}

func (d *Deployer) waitForRollout(ctx context.Context, ns, name string) error {
    for {
        dep, err := d.clientset.AppsV1().Deployments(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        desired := *dep.Spec.Replicas
        if dep.Status.UpdatedReplicas == desired &&
            dep.Status.ReadyReplicas == desired &&
            dep.Status.AvailableReplicas == desired {
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
        }
    }
}

func (d *Deployer) rollback(ctx context.Context, ns, name string) {
    retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := d.clientset.AppsV1().Deployments(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        if dep.Spec.Template.Annotations == nil {
            dep.Spec.Template.Annotations = map[string]string{}
        }
        dep.Spec.Template.Annotations["rollback"] =
time.Now().Format(time.RFC3339)
        _, err = d.clientset.AppsV1().Deployments(ns).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
```

```
    })
}
```

---

4. Cluster Health Dashboard

```go
type ClusterHealth struct {
    Nodes       NodeHealth       `json:"nodes"`
    Pods        PodHealth        `json:"pods"`
    Deployments DeploymentHealth `json:"deployments"`
    Timestamp   string           `json:"timestamp"`
}

func clusterHealthReport(ctx context.Context, clientset *kubernetes.Clientset)
(*ClusterHealth, error) {
    health := &ClusterHealth{Timestamp: time.Now().Format(time.RFC3339)}

    // Nodes
    nodes, err := clientset.CoreV1().Nodes().List(ctx, metav1.ListOptions{})
    if err != nil {
        return nil, err
    }
    health.Nodes.Total = len(nodes.Items)
    for _, node := range nodes.Items {
        for _, cond := range node.Status.Conditions {
            if cond.Type == corev1.NodeReady && cond.Status ==
corev1.ConditionTrue {
                health.Nodes.Ready++
            }
        }
    }

    // Pods (all namespaces)
    pods, err := clientset.CoreV1().Pods("").List(ctx, metav1.ListOptions{})
    if err != nil {
        return nil, err
    }
    health.Pods.Total = len(pods.Items)
    for _, pod := range pods.Items {
        switch pod.Status.Phase {
        case corev1.PodRunning:
            health.Pods.Running++
        case corev1.PodFailed:
```

```
            health.Pods.Failed++
        case corev1.PodPending:
            health.Pods.Pending++
        }
    }

    // Deployments
    deps, err := clientset.AppsV1().Deployments("").List(ctx,
metav1.ListOptions{})
    if err != nil {
        return nil, err
    }
    health.Deployments.Total = len(deps.Items)
    for _, dep := range deps.Items {
        if dep.Status.ReadyReplicas == *dep.Spec.Replicas {
            health.Deployments.Healthy++
        }
    }

    return health, nil
}
```

---

5. Graceful Shutdown

go

```
import (
    "os"
    "os/signal"
    "syscall"
)

func main() {
    config, _ := rest.InClusterConfig()
    clientset, _ := kubernetes.NewForConfig(config)
    factory := informers.NewSharedInformerFactory(clientset, 30*time.Second)
    ctrl := NewPodController(clientset, factory)

    // Context cancelled on SIGINT / SIGTERM
    ctx, cancel := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    factory.Start(ctx.Done())
```

```
    ctrl.Run(ctx, 2)

    log.Println("Controller stopped cleanly")
}
```

Production Checklist

Concern	Solution
Authentication	`rest.InClusterConfig()` in production pods
Least privilege	ServiceAccount with minimal RBAC
Conflict retry	Always use `retry.RetryOnConflict` for updates
Context timeouts	Every API call gets a context with deadline
Rate limiting	Set `config.QPS` and `config.Burst` appropriately
Leader election	Use `leaderelection` for multi-replica controllers
Informer cache	Use informers + listers — avoid direct API calls in hot paths
Graceful shutdown	`signal.NotifyContext` + `factory.Shutdown()`
Error logging	Log with structured fields (namespace, name, reason)
Health probes	Expose `/healthz` and `/readyz` endpoints
Metrics	Use `k8s.io/component-base/metrics` for controller metrics

k8s-client-go — Master Cheatsheet

Install

```
go get k8s.io/client-go@v0.29.0
go get k8s.io/api@v0.29.0
go get k8s.io/apimachinery@v0.29.0
go mod tidy
```

Connect

```go
import (
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/rest"
    "k8s.io/client-go/tools/clientcmd"
)

// From kubeconfig (dev)
config, _ := clientcmd.BuildConfigFromFlags("", filepath.Join(os.UserHomeDir(),
".kube", "config"))
clientset, _ := kubernetes.NewForConfig(config)

// In-cluster (Pod inside K8s)
config, _ = rest.InClusterConfig()
clientset, _ = kubernetes.NewForConfig(config)

// Auto-detect (try in-cluster, fall back to kubeconfig)
config, err := rest.InClusterConfig()
if err != nil {
    config, err = clientcmd.BuildConfigFromFlags("", kubeconfig)
}
clientset, _ = kubernetes.NewForConfig(config)

// Specific context
overrides := &clientcmd.ConfigOverrides{CurrentContext: "my-cluster"}
config, _ = clientcmd.NewNonInteractiveDeferredLoadingClientConfig(
    clientcmd.NewDefaultClientConfigLoadingRules(), overrides,
).ClientConfig()

// Tune QPS
config.QPS = 100
config.Burst = 200
```

Pods

```go
// List
clientset.CoreV1().Pods("default").List(ctx, metav1.ListOptions{})
clientset.CoreV1().Pods("").List(ctx, metav1.ListOptions{})
```

```
// all namespaces
clientset.CoreV1().Pods("ns").List(ctx, metav1.ListOptions{LabelSelector:
"app=nginx"})
clientset.CoreV1().Pods("ns").List(ctx, metav1.ListOptions{FieldSelector:
"status.phase=Running"})

// Get
clientset.CoreV1().Pods("default").Get(ctx, "my-pod", metav1.GetOptions{})

// Create
clientset.CoreV1().Pods("default").Create(ctx, &corev1.Pod{...},
metav1.CreateOptions{})

// Delete
clientset.CoreV1().Pods("default").Delete(ctx, "my-pod",
metav1.DeleteOptions{})
clientset.CoreV1().Pods("default").Delete(ctx, "my-pod",
metav1.DeleteOptions{GracePeriodSeconds: int64Ptr(0)})

// Logs
req := clientset.CoreV1().Pods("default").GetLogs("my-pod",
&corev1.PodLogOptions{})
req := clientset.CoreV1().Pods("default").GetLogs("my-pod",
&corev1.PodLogOptions{TailLines: int64Ptr(100)})
req := clientset.CoreV1().Pods("default").GetLogs("my-pod",
&corev1.PodLogOptions{Previous: true})
req := clientset.CoreV1().Pods("default").GetLogs("my-pod",
&corev1.PodLogOptions{Container: "sidecar"})
stream, _ := req.Stream(ctx)
defer stream.Close()
io.Copy(os.Stdout, stream)

// Watch
watcher, _ := clientset.CoreV1().Pods("default").Watch(ctx,
metav1.ListOptions{})
for event := range watcher.ResultChan() {
    pod := event.Object.(*corev1.Pod)
    fmt.Printf("[%s] %s\n", event.Type, pod.Name)
}

// Read pod fields
pod.Name
pod.Namespace
pod.Status.Phase          // corev1.PodRunning | PodPending | PodFailed |
PodSucceeded
pod.Status.PodIP
```

```
pod.Spec.NodeName
pod.Status.ContainerStatuses[0].RestartCount
pod.Status.ContainerStatuses[0].State.Waiting.Reason   // CrashLoopBackOff,
ImagePullBackOff
```

Deployments

go

```
// List / Get
clientset.AppsV1().Deployments("ns").List(ctx, metav1.ListOptions{})
clientset.AppsV1().Deployments("ns").Get(ctx, "name", metav1.GetOptions{})

// Create
clientset.AppsV1().Deployments("ns").Create(ctx, &appsv1.Deployment{...},
metav1.CreateOptions{})

// Scale (with retry)
retry.RetryOnConflict(retry.DefaultRetry, func() error {
    d, _ := clientset.AppsV1().Deployments("ns").Get(ctx, "name",
metav1.GetOptions{})
    d.Spec.Replicas = int32Ptr(5)
    _, err := clientset.AppsV1().Deployments("ns").Update(ctx, d,
metav1.UpdateOptions{})
    return err
})

// Update image
retry.RetryOnConflict(retry.DefaultRetry, func() error {
    d, _ := clientset.AppsV1().Deployments("ns").Get(ctx, "name",
metav1.GetOptions{})
    d.Spec.Template.Spec.Containers[0].Image = "myapp:2.0"
    _, err := clientset.AppsV1().Deployments("ns").Update(ctx, d,
metav1.UpdateOptions{})
    return err
})

// Rollout restart
retry.RetryOnConflict(retry.DefaultRetry, func() error {
    d, _ := clientset.AppsV1().Deployments("ns").Get(ctx, "name",
metav1.GetOptions{})
    if d.Spec.Template.Annotations == nil { d.Spec.Template.Annotations =
map[string]string{} }
    d.Spec.Template.Annotations["kubectl.kubernetes.io/restartedAt"] =
```

```
time.Now().Format(time.RFC3339)
    _, err := clientset.AppsV1().Deployments("ns").Update(ctx, d,
metav1.UpdateOptions{})
    return err
})

// Delete
clientset.AppsV1().Deployments("ns").Delete(ctx, "name",
metav1.DeleteOptions{PropagationPolicy: &foreground})

// Read status
dep.Spec.Replicas
dep.Status.ReadyReplicas
dep.Status.UpdatedReplicas
dep.Status.AvailableReplicas
```

Services

```go
clientset.CoreV1().Services("ns").List(ctx, metav1.ListOptions{})
clientset.CoreV1().Services("ns").Get(ctx, "name", metav1.GetOptions{})
clientset.CoreV1().Services("ns").Create(ctx, &corev1.Service{
    Spec: corev1.ServiceSpec{
        Type:     corev1.ServiceTypeClusterIP,    // NodePort | LoadBalancer |
ExternalName
        Selector: map[string]string{"app": "x"},
        Ports:    []corev1.ServicePort{{Port: 80, TargetPort:
intstr.FromInt(8080)}},
    },
}, metav1.CreateOptions{})
clientset.CoreV1().Services("ns").Delete(ctx, "name", metav1.DeleteOptions{})

svc.Spec.ClusterIP
svc.Spec.Type
svc.Status.LoadBalancer.Ingress[0].IP
```

ConfigMaps

```go
clientset.CoreV1().ConfigMaps("ns").List(ctx, metav1.ListOptions{})
clientset.CoreV1().ConfigMaps("ns").Get(ctx, "name", metav1.GetOptions{})
cm.Data["KEY"]

clientset.CoreV1().ConfigMaps("ns").Create(ctx, &corev1.ConfigMap{
    Data: map[string]string{"KEY": "value"},
}, metav1.CreateOptions{})

retry.RetryOnConflict(retry.DefaultRetry, func() error {
    cm, _ := clientset.CoreV1().ConfigMaps("ns").Get(ctx, "name",
metav1.GetOptions{})
    cm.Data["KEY"] = "new"
    _, err := clientset.CoreV1().ConfigMaps("ns").Update(ctx, cm,
metav1.UpdateOptions{})
    return err
})

clientset.CoreV1().ConfigMaps("ns").Delete(ctx, "name", metav1.DeleteOptions{})
```

Secrets

```go
clientset.CoreV1().Secrets("ns").Get(ctx, "name", metav1.GetOptions{})
string(secret.Data["password"])   // already decoded — no base64 needed

clientset.CoreV1().Secrets("ns").Create(ctx, &corev1.Secret{
    Type:       corev1.SecretTypeOpaque,
    StringData: map[string]string{"password": "secret"},  // auto-encoded
}, metav1.CreateOptions{})

clientset.CoreV1().Secrets("ns").Delete(ctx, "name", metav1.DeleteOptions{})
```

Namespaces & RBAC

```go
clientset.CoreV1().Namespaces().List(ctx, metav1.ListOptions{})
clientset.CoreV1().Namespaces().Create(ctx, &corev1.Namespace{ObjectMeta:
metav1.ObjectMeta{Name: "ns"}}, metav1.CreateOptions{})
clientset.CoreV1().Namespaces().Delete(ctx, "ns", metav1.DeleteOptions{})
```

```
clientset.RbacV1().Roles("ns").Create(ctx, &rbacv1.Role{Rules:
[]rbacv1.PolicyRule{{APIGroups: []string{""}, Resources: []string{"pods"},
Verbs: []string{"get","list"}}}}, metav1.CreateOptions{})
clientset.RbacV1().ClusterRoles().Create(ctx, &rbacv1.ClusterRole{...},
metav1.CreateOptions{})
clientset.RbacV1().RoleBindings("ns").Create(ctx, &rbacv1.RoleBinding{Subjects:
[...], RoleRef: ...}, metav1.CreateOptions{})
clientset.RbacV1().ClusterRoleBindings().Create(ctx,
&rbacv1.ClusterRoleBinding{...}, metav1.CreateOptions{})
clientset.CoreV1().ServiceAccounts("ns").Create(ctx,
&corev1.ServiceAccount{ObjectMeta: metav1.ObjectMeta{Name: "sa"}},
metav1.CreateOptions{})
```

PVC / Storage

go

```
clientset.CoreV1().PersistentVolumeClaims("ns").List(ctx, metav1.ListOptions{})
clientset.CoreV1().PersistentVolumeClaims("ns").Create(ctx,
&corev1.PersistentVolumeClaim{
    Spec: corev1.PersistentVolumeClaimSpec{
        AccessModes:
[]corev1.PersistentVolumeAccessMode{corev1.ReadWriteOnce},
        StorageClassName: &storageClass,
        Resources:        corev1.ResourceRequirements{Requests:
corev1.ResourceList{corev1.ResourceStorage: resource.MustParse("10Gi")}},
    },
}, metav1.CreateOptions{})
pvc.Status.Phase   // corev1.ClaimBound | ClaimPending | ClaimLost
clientset.CoreV1().PersistentVolumes().List(ctx, metav1.ListOptions{})
clientset.StorageV1().StorageClasses().List(ctx, metav1.ListOptions{})
```

Jobs & CronJobs

go

```
clientset.BatchV1().Jobs("ns").List(ctx, metav1.ListOptions{})
clientset.BatchV1().Jobs("ns").Create(ctx, &batchv1.Job{...},
metav1.CreateOptions{})
job.Status.Succeeded
```

```
job.Status.Failed
job.Status.Active
clientset.BatchV1().Jobs("ns").Delete(ctx, "name",
metav1.DeleteOptions{PropagationPolicy: &foreground})

clientset.BatchV1().CronJobs("ns").List(ctx, metav1.ListOptions{})
clientset.BatchV1().CronJobs("ns").Create(ctx, &batchv1.CronJob{Spec:
batchv1.CronJobSpec{Schedule: "0 * * * *", ...}}, metav1.CreateOptions{})
cj.Spec.Schedule
cj.Status.LastScheduleTime
```

Informers

```go
factory := informers.NewSharedInformerFactory(clientset, 30*time.Second)
podInformer := factory.Core().V1().Pods().Informer()

podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc:    func(obj interface{}) { pod := obj.(*corev1.Pod);
fmt.Println("ADDED:", pod.Name) },
    UpdateFunc: func(_, obj interface{}) { pod := obj.(*corev1.Pod);
fmt.Println("UPDATED:", pod.Name) },
    DeleteFunc: func(obj interface{}) { /* handle tombstone */ },
})

factory.Start(stopCh)
cache.WaitForCacheSync(stopCh, podInformer.HasSynced)

// Lister (no API call — reads from cache)
lister := factory.Core().V1().Pods().Lister()
pods, _ := lister.Pods("default").List(labels.Everything())
pod, _ := lister.Pods("default").Get("my-pod")
```

Dynamic Client (CRDs)

```go
dynClient, _ := dynamic.NewForConfig(config)

gvr := schema.GroupVersionResource{Group: "cert-manager.io", Version: "v1",
```

```
Resource: "certificates"}

dynClient.Resource(gvr).Namespace("default").List(ctx, metav1.ListOptions{})
dynClient.Resource(gvr).Namespace("default").Get(ctx, "name",
metav1.GetOptions{})
dynClient.Resource(gvr).Namespace("default").Create(ctx,
&unstructured.Unstructured{Object: map[string]interface{}{...}},
metav1.CreateOptions{})
dynClient.Resource(gvr).Namespace("default").Delete(ctx, "name",
metav1.DeleteOptions{})

// Read fields
name, _, _ := unstructured.NestedString(obj.Object, "spec", "secretName")
unstructured.SetNestedField(obj.Object, "value", "spec", "field")
```

Error Handling

```go
import "k8s.io/apimachinery/pkg/api/errors"

errors.IsNotFound(err)       // 404
errors.IsAlreadyExists(err)  // 409
errors.IsConflict(err)       // 409 (stale resourceVersion)
errors.IsForbidden(err)      // 403
errors.IsUnauthorized(err)   // 401
errors.IsInvalid(err)        // 422
errors.IsTooManyRequests(err)// 429

// Retry on conflict (always use for updates)
retry.RetryOnConflict(retry.DefaultRetry, func() error { ... })
```

Helper Functions

```go
func int32Ptr(i int32) *int32 { return &i }
func int64Ptr(i int64) *int64 { return &i }
func boolPtr(b bool) *bool    { return &b }
func stringPtr(s string) *string { return &s }
```

```
// Propagation
foreground := metav1.DeletePropagationForeground
background := metav1.DeletePropagationBackground
```

Key Imports

go

```
import (
    "context"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "time"

    appsv1  "k8s.io/api/apps/v1"
    batchv1 "k8s.io/api/batch/v1"
    corev1  "k8s.io/api/core/v1"
    netv1   "k8s.io/api/networking/v1"
    rbacv1  "k8s.io/api/rbac/v1"

    "k8s.io/apimachinery/pkg/api/errors"
    "k8s.io/apimachinery/pkg/api/resource"
    metav1  "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/util/intstr"
    "k8s.io/apimachinery/pkg/runtime/schema"

    "k8s.io/client-go/dynamic"
    "k8s.io/client-go/informers"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/rest"
    "k8s.io/client-go/tools/cache"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/util/retry"
    "k8s.io/client-go/util/workqueue"
)
```

k8s-client-go — Code Snippets Cheatsheet

Copy-paste patterns for real-world tasks.

Auto-detect Config (In-cluster or kubeconfig)

```go
func buildConfig() (*rest.Config, error) {
    if config, err := rest.InClusterConfig(); err == nil {
        return config, nil
    }
    home, _ := os.UserHomeDir()
    return clientcmd.BuildConfigFromFlags("", filepath.Join(home, ".kube",
"config"))
}
```

Create or Update (Apply)

```go
func applyDeployment(ctx context.Context, cs *kubernetes.Clientset, dep
*appsv1.Deployment) (string, error) {
    _, err := cs.AppsV1().Deployments(dep.Namespace).Get(ctx, dep.Name,
metav1.GetOptions{})
    if errors.IsNotFound(err) {
        _, err = cs.AppsV1().Deployments(dep.Namespace).Create(ctx, dep,
metav1.CreateOptions{})
        return "created", err
    }
    if err != nil {
        return "", err
    }
    err = retry.RetryOnConflict(retry.DefaultRetry, func() error {
        existing, err := cs.AppsV1().Deployments(dep.Namespace).Get(ctx,
dep.Name, metav1.GetOptions{})
        if err != nil {
            return err
        }
        dep.ResourceVersion = existing.ResourceVersion
        _, err = cs.AppsV1().Deployments(dep.Namespace).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
    return "updated", err
}
```

Delete If Exists

```go
func deleteIfExists(ctx context.Context, cs *kubernetes.Clientset, ns, name
string) error {
    err := cs.AppsV1().Deployments(ns).Delete(ctx, name,
metav1.DeleteOptions{})
    if errors.IsNotFound(err) {
        return nil
    }
    return err
}
```

Wait for Pod Running

```go
func waitForPodRunning(ctx context.Context, cs *kubernetes.Clientset, ns, name
string) error {
    for {
        pod, err := cs.CoreV1().Pods(ns).Get(ctx, name, metav1.GetOptions{})
        if err != nil {
            return err
        }
        if pod.Status.Phase == corev1.PodRunning {
            return nil
        }
        if pod.Status.Phase == corev1.PodFailed {
            return fmt.Errorf("pod %s failed", name)
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(2 * time.Second):
        }
    }
}
```

Wait for Deployment Rollout

```go
func waitForDeployment(ctx context.Context, cs *kubernetes.Clientset, ns, name string) error {
    for {
        dep, err := cs.AppsV1().Deployments(ns).Get(ctx, name, metav1.GetOptions{})
        if err != nil {
            return err
        }
        d := *dep.Spec.Replicas
        if dep.Status.UpdatedReplicas == d && dep.Status.ReadyReplicas == d && dep.Status.AvailableReplicas == d {
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
        }
    }
}
```

Wait for Job Completion

```go
func waitForJob(ctx context.Context, cs *kubernetes.Clientset, ns, name string) (string, error) {
    for {
        job, err := cs.BatchV1().Jobs(ns).Get(ctx, name, metav1.GetOptions{})
        if err != nil {
            return "", err
        }
        if job.Status.Succeeded > 0 {
            return "succeeded", nil
        }
        if job.Status.Failed > *job.Spec.BackoffLimit {
            return "failed", fmt.Errorf("job failed after %d attempts", job.Status.Failed)
        }
        select {
        case <-ctx.Done():
```

```
            return "timeout", ctx.Err()
        case <-time.After(5 * time.Second):
        }
    }
}
```

Wait for PVC Bound

```go
func waitForPVC(ctx context.Context, cs *kubernetes.Clientset, ns, name string)
error {
    for {
        pvc, err := cs.CoreV1().PersistentVolumeClaims(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        if pvc.Status.Phase == corev1.ClaimBound {
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(3 * time.Second):
        }
    }
}
```

Wait for LoadBalancer IP

```go
func waitForLBIP(ctx context.Context, cs *kubernetes.Clientset, ns, name
string) (string, error) {
    for {
        svc, err := cs.CoreV1().Services(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return "", err
        }
```

```
        for _, ing := range svc.Status.LoadBalancer.Ingress {
            if ing.IP != "" {
                return ing.IP, nil
            }
            if ing.Hostname != "" {
                return ing.Hostname, nil
            }
        }
        select {
        case <-ctx.Done():
            return "", ctx.Err()
        case <-time.After(5 * time.Second):
        }
    }
}
```

Scale Deployment

```go
func scale(ctx context.Context, cs *kubernetes.Clientset, ns, name string,
replicas int32) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := cs.AppsV1().Deployments(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        dep.Spec.Replicas = &replicas
        _, err = cs.AppsV1().Deployments(ns).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
}

scale(ctx, clientset, "default", "my-app", 0)  // scale to zero
scale(ctx, clientset, "default", "my-app", 5)  // scale up
```

Update Image

```go
func updateImage(ctx context.Context, cs *kubernetes.Clientset, ns, dep,
container, image string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        d, err := cs.AppsV1().Deployments(ns).Get(ctx, dep,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        for i, c := range d.Spec.Template.Spec.Containers {
            if c.Name == container {
                d.Spec.Template.Spec.Containers[i].Image = image
            }
        }
        _, err = cs.AppsV1().Deployments(ns).Update(ctx, d,
metav1.UpdateOptions{})
        return err
    })
}
```

Rollout Restart

```go
func rolloutRestart(ctx context.Context, cs *kubernetes.Clientset, ns, name
string) error {
    return retry.RetryOnConflict(retry.DefaultRetry, func() error {
        dep, err := cs.AppsV1().Deployments(ns).Get(ctx, name,
metav1.GetOptions{})
        if err != nil {
            return err
        }
        if dep.Spec.Template.Annotations == nil {
            dep.Spec.Template.Annotations = map[string]string{}
        }
        dep.Spec.Template.Annotations["kubectl.kubernetes.io/restartedAt"] =
time.Now().Format(time.RFC3339)
        _, err = cs.AppsV1().Deployments(ns).Update(ctx, dep,
metav1.UpdateOptions{})
        return err
    })
}
```

Get Pod Logs as String

```go
func getPodLogs(ctx context.Context, cs *kubernetes.Clientset, ns, name string,
tailLines int64) (string, error) {
    req := cs.CoreV1().Pods(ns).GetLogs(name, &corev1.PodLogOptions{TailLines:
&tailLines})
    stream, err := req.Stream(ctx)
    if err != nil {
        return "", err
    }
    defer stream.Close()
    data, err := io.ReadAll(stream)
    return string(data), err
}
```

Get Pods for a Deployment

```go
func getDeploymentPods(ctx context.Context, cs *kubernetes.Clientset, ns, name
string) ([]corev1.Pod, error) {
    dep, err := cs.AppsV1().Deployments(ns).Get(ctx, name, metav1.GetOptions{})
    if err != nil {
        return nil, err
    }
    selector := labels.Set(dep.Spec.Selector.MatchLabels).String()
    pods, err := cs.CoreV1().Pods(ns).List(ctx,
metav1.ListOptions{LabelSelector: selector})
    if err != nil {
        return nil, err
    }
    return pods.Items, nil
}
```

Provision Tenant Namespace

```go
func provisionTenant(ctx context.Context, cs *kubernetes.Clientset, team
string) error {
```

```
    ns := team + "-workspace"
    cs.CoreV1().Namespaces().Create(ctx, &corev1.Namespace{
        ObjectMeta: metav1.ObjectMeta{Name: ns, Labels:
map[string]string{"team": team}},
    }, metav1.CreateOptions{})

    cs.CoreV1().ServiceAccounts(ns).Create(ctx, &corev1.ServiceAccount{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-sa"},
    }, metav1.CreateOptions{})

    cs.RbacV1().Roles(ns).Create(ctx, &rbacv1.Role{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-role"},
        Rules: []rbacv1.PolicyRule{{
            APIGroups: []string{"", "apps", "batch"}, Resources: []string{"*"},
Verbs: []string{"*"},
        }},
    }, metav1.CreateOptions{})

    cs.RbacV1().RoleBindings(ns).Create(ctx, &rbacv1.RoleBinding{
        ObjectMeta: metav1.ObjectMeta{Name: team + "-binding"},
        Subjects:   []rbacv1.Subject{{Kind: "ServiceAccount", Name: team + "-
sa", Namespace: ns}},
        RoleRef:    rbacv1.RoleRef{APIGroup: "rbac.authorization.k8s.io", Kind:
"Role", Name: team + "-role"},
    }, metav1.CreateOptions{})

    return nil
}
```

Retry with Exponential Backoff

go

```
func withRetry(ctx context.Context, maxSteps int, fn func() error) error {
    backoff := wait.Backoff{
        Duration: 500 * time.Millisecond,
        Factor:   2.0,
        Jitter:   0.1,
        Steps:    maxSteps,
    }
    return wait.ExponentialBackoffWithContext(ctx, backoff, func(ctx
context.Context) (bool, error) {
        err := fn()
        if err == nil {
```

```
            return true, nil
        }
        if errors.IsServerTimeout(err) || errors.IsServiceUnavailable(err) ||
errors.IsTooManyRequests(err) {
            return false, nil  // retry
        }
        return false, err  // permanent error
    })
}
```

Cluster Pod Summary

```go
func podSummary(ctx context.Context, cs *kubernetes.Clientset) {
    pods, _ := cs.CoreV1().Pods("").List(ctx, metav1.ListOptions{})
    phases := map[corev1.PodPhase]int{}
    for _, p := range pods.Items {
        phases[p.Status.Phase]++
    }
    for phase, count := range phases {
        fmt.Printf("%-12s %d\n", phase, count)
    }
}
```

Watch for CrashLoopBackOff

```go
func watchCrashLoops(ctx context.Context, cs *kubernetes.Clientset) {
    factory := informers.NewSharedInformerFactory(cs, 30*time.Second)

factory.Core().V1().Pods().Informer().AddEventHandler(cache.ResourceEventHandle
rFuncs{
        UpdateFunc: func(_, obj interface{}) {
            pod := obj.(*corev1.Pod)
            for _, cs := range pod.Status.ContainerStatuses {
                if cs.State.Waiting != nil && cs.State.Waiting.Reason ==
"CrashLoopBackOff" {
                    fmt.Printf("[ALERT] CrashLoopBackOff: pod=%s container=%s
restarts=%d\n",
```

```
                        pod.Name, cs.Name, cs.RestartCount)
                }
            }
        },
    })
    factory.Start(ctx.Done())
    <-ctx.Done()
}
```

k8s-client-go — kubectl vs Go Client Cheatsheet

Side-by-side reference.

Pods

kubectl	Go (client-go)
`kubectl get pods`	`cs.CoreV1().Pods("default").List(ctx, metav1.ListOptions{})`
`kubectl get pods -A`	`cs.CoreV1().Pods("").List(ctx, metav1.ListOptions{})`
`kubectl get pods -n staging`	`cs.CoreV1().Pods("staging").List(ctx, metav1.ListOptions{})`
`kubectl get pods -l app=nginx`	`cs.CoreV1().Pods("ns").List(ctx, metav1.ListOptions{LabelSelector: "app=nginx"})`
`kubectl get pod my-pod`	`cs.CoreV1().Pods("ns").Get(ctx, "my-pod", metav1.GetOptions{})`
`kubectl logs my-pod`	`cs.CoreV1().Pods("ns").GetLogs("my-pod", &corev1.PodLogOptions{})`
`kubectl logs my-pod --tail=100`	`&corev1.PodLogOptions{TailLines: int64Ptr(100)}`
`kubectl logs my-pod -p`	`&corev1.PodLogOptions{Previous: true}`

kubectl	Go (client-go)
`kubectl logs my-pod -c sidecar`	`&corev1.PodLogOptions{Container: "sidecar"}`
`kubectl logs -f my-pod`	`&corev1.PodLogOptions{Follow: true}`
`kubectl delete pod my-pod`	`cs.CoreV1().Pods("ns").Delete(ctx, "my-pod", metav1.DeleteOptions{})`
`kubectl delete pod my-pod --grace-period=0`	`metav1.DeleteOptions{GracePeriodSeconds: int64Ptr(0)}`
`kubectl get pods -w`	`cs.CoreV1().Pods("ns").Watch(ctx, metav1.ListOptions{})`

Deployments

kubectl	Go (client-go)
`kubectl get deployments`	`cs.AppsV1().Deployments("ns").List(ctx, metav1.ListOptions{})`
`kubectl get deploy my-app`	`cs.AppsV1().Deployments("ns").Get(ctx, "my-app", metav1.GetOptions{})`
`kubectl scale deploy my-app --replicas=5`	`retry.RetryOnConflict(...) { d.Spec.Replicas = int32Ptr(5) }`
`kubectl set image deploy/my-app app=myapp:2.0`	`retry.RetryOnConflict(...) { d.Spec.Template.Spec.Containers[0].Image = "myapp:2.0" }`
`kubectl rollout restart deploy/my-app`	`d.Spec.Template.Annotations["kubectl.kubernetes.io/restartedAt"] = time.Now()...`
`kubectl rollout status deploy/my-app`	`waitForDeployment(ctx, cs, "ns", "my-app")`

kubectl	Go (client-go)
`kubectl delete deploy my-app`	`cs.AppsV1().Deployments("ns").Delete(ctx, "my-app", metav1.DeleteOptions{})`
`kubectl apply -f deploy.yaml`	`applyDeployment(ctx, cs, dep)` (create-or-update pattern)

Services

kubectl	Go (client-go)
`kubectl get services`	`cs.CoreV1().Services("ns").List(ctx, metav1.ListOptions{})`
`kubectl get svc my-svc`	`cs.CoreV1().Services("ns").Get(ctx, "my-svc", metav1.GetOptions{})`
`kubectl delete svc my-svc`	`cs.CoreV1().Services("ns").Delete(ctx, "my-svc", metav1.DeleteOptions{})`

ConfigMaps & Secrets

kubectl	Go (client-go)
`kubectl get configmaps`	`cs.CoreV1().ConfigMaps("ns").List(ctx, metav1.ListOptions{})`
`kubectl get cm app-config -o json`	`cs.CoreV1().ConfigMaps("ns").Get(ctx, "app-config", metav1.GetOptions{})`
`kubectl create cm app-config --from-literal=K=V`	`cs.CoreV1().ConfigMaps("ns").Create(ctx, &corev1.ConfigMap{Data: map[string]string{"K":"V"}}, ...)`

kubectl	Go (client-go)
`kubectl delete cm app-config`	`cs.CoreV1().ConfigMaps("ns").Delete(ctx, "app-config", metav1.DeleteOptions{})`
`kubectl get secrets`	`cs.CoreV1().Secrets("ns").List(ctx, metav1.ListOptions{})`
`kubectl get secret db-creds -o jsonpath='{.data.password}'`	`string(secret.Data["password"])` (already decoded)
`kubectl create secret generic db --from-literal=p=x`	`cs.CoreV1().Secrets("ns").Create(ctx, &corev1.Secret{StringData: map[string]string{"p":"x"}}, ...)`
`kubectl delete secret db-creds`	`cs.CoreV1().Secrets("ns").Delete(ctx, "db-creds", metav1.DeleteOptions{})`

Namespaces

kubectl	Go (client-go)
`kubectl get namespaces`	`cs.CoreV1().Namespaces().List(ctx, metav1.ListOptions{})`
`kubectl create namespace team`	`cs.CoreV1().Namespaces().Create(ctx, &corev1.Namespace{ObjectMeta: metav1.ObjectMeta{Name: "team"}}, ...)`
`kubectl delete namespace team`	`cs.CoreV1().Namespaces().Delete(ctx, "team", metav1.DeleteOptions{})`

Jobs & CronJobs

kubectl	Go (client-go)
`kubectl get jobs`	`cs.BatchV1().Jobs("ns").List(ctx, metav1.ListOptions{})`
`kubectl delete job my-job`	`cs.BatchV1().Jobs("ns").Delete(ctx, "my-job", metav1.DeleteOptions{PropagationPolicy: &foreground})`
`kubectl get cronjobs`	`cs.BatchV1().CronJobs("ns").List(ctx, metav1.ListOptions{})`
`kubectl patch cj my-cj -p '{"spec": {"suspend":true}}'`	`cs.BatchV1().CronJobs("ns").Patch(ctx, "name", types.MergePatchType, suspendJSON, metav1.PatchOptions{})`

StatefulSets & DaemonSets

kubectl	Go (client-go)
`kubectl get statefulsets`	`cs.AppsV1().StatefulSets("ns").List(ctx, metav1.ListOptions{})`
`kubectl scale sts mysql --replicas=5`	`retry.RetryOnConflict(...) { sts.Spec.Replicas = int32Ptr(5) }`
`kubectl get daemonsets`	`cs.AppsV1().DaemonSets("ns").List(ctx, metav1.ListOptions{})`

Ingress

kubectl	Go (client-go)
`kubectl get ingress`	`cs.NetworkingV1().Ingresses("ns").List(ctx, metav1.ListOptions{})`

kubectl	Go (client-go)
`kubectl delete ingress my-ing`	`cs.NetworkingV1().Ingresses("ns").Delete(ctx, "my-ing", metav1.DeleteOptions{})`

RBAC

kubectl	Go (client-go)
`kubectl get roles`	`cs.RbacV1().Roles("ns").List(ctx, metav1.ListOptions{})`
`kubectl get rolebindings`	`cs.RbacV1().RoleBindings("ns").List(ctx, metav1.ListOptions{})`
`kubectl get clusterroles`	`cs.RbacV1().ClusterRoles().List(ctx, metav1.ListOptions{})`
`kubectl get clusterrolebindings`	`cs.RbacV1().ClusterRoleBindings().List(ctx, metav1.ListOptions{})`
`kubectl get serviceaccounts`	`cs.CoreV1().ServiceAccounts("ns").List(ctx, metav1.ListOptions{})`

Storage

kubectl	Go (client-go)
`kubectl get pvc`	`cs.CoreV1().PersistentVolumeClaims("ns").List(ctx, metav1.ListOptions{})`
`kubectl get pv`	`cs.CoreV1().PersistentVolumes().List(ctx, metav1.ListOptions{})`

kubectl	Go (client-go)
`kubectl get storageclass`	`cs.StorageV1().StorageClasses().List(ctx, metav1.ListOptions{})`

Nodes

kubectl	Go (client-go)
`kubectl get nodes`	`cs.CoreV1().Nodes().List(ctx, metav1.ListOptions{})`
`kubectl get node node-1`	`cs.CoreV1().Nodes().Get(ctx, "node-1", metav1.GetOptions{})`

Events

kubectl	Go (client-go)
`kubectl get events`	`cs.CoreV1().Events("ns").List(ctx, metav1.ListOptions{})`
`kubectl get events --field-selector type=Warning`	`cs.CoreV1().Events("ns").List(ctx, metav1.ListOptions{FieldSelector: "type=Warning"})`
`kubectl get events -w`	Informer on Events resource

Watch

kubectl	Go (client-go)
`kubectl get pods -w`	`cs.CoreV1().Pods("ns").Watch(ctx, metav1.ListOptions{})`
`kubectl get deployments -w`	`cs.AppsV1().Deployments("ns").Watch(ctx, metav1.ListOptions{})`
Efficient + cached watching	`informers.NewSharedInformerFactory(cs, 30*time.Second)`

Error Code Reference

HTTP Code	kubectl output	Go check
200	Success	`err == nil`
401	Unauthorized	`errors.IsUnauthorized(err)`
403	Forbidden	`errors.IsForbidden(err)`
404	NotFound	`errors.IsNotFound(err)`
409	Conflict / AlreadyExists	`errors.IsConflict(err)` or `errors.IsAlreadyExists(err)`
422	Invalid	`errors.IsInvalid(err)`
429	TooManyRequests	`errors.IsTooManyRequests(err)`
503	ServiceUnavailable	`errors.IsServiceUnavailable(err)`